Project Name - Deep Learning _ CSAT Project

Project Type - EDA/Regression/Classification/Unsupervised

Contribution - Individual

Submitted by - ***SUBHAM KUNDU***

Project Summary -

This project focuses on analyzing an e-commerce customer support dataset and building predictive models to understand patterns in customer service performance. The main goal of the project was to explore the dataset, preprocess the data properly, test different hypotheses, and build machine learning models that can make reliable predictions based on the available features.

The project started with data exploration and visualization to understand the structure and characteristics of the dataset. Various exploratory data analysis techniques were applied to identify relationships between variables and detect possible trends in the data. Visualizations such as pair plots and other charts helped in understanding how different features interact with each other. These initial analyses provided useful insights that guided the next steps of the project.

After understanding the dataset, several data preprocessing and feature engineering techniques were applied. Missing values were handled using suitable imputation techniques to ensure that the dataset remained consistent and usable for modeling. Outliers were identified and treated to reduce their impact on the model's performance. Categorical variables were converted into numerical form through encoding methods so that machine learning algorithms could process them effectively. Additionally, scaling techniques were applied to numerical features to ensure that all variables were on a similar scale, which helps improve the performance and stability of many machine learning algorithms.

The project also involved text preprocessing and normalization steps where necessary. Techniques such as expanding contractions, converting text to lowercase, removing punctuation, eliminating stopwords, tokenization, and lemmatization were used to clean and standardize textual data. These steps help reduce noise in the dataset and make the information more useful for analysis and modeling.

After preprocessing, feature selection and transformation were performed to reduce unnecessary or highly correlated variables. This helps improve model efficiency and reduces the risk of overfitting. The dataset was then divided into training and testing sets using a suitable data splitting ratio so that the models could be trained on one portion of the data and evaluated on unseen data.

To build predictive models, three different algorithms were implemented. The first model was Linear Regression, which served as a baseline model to understand the basic relationship between the features and the target variable. The second model used was Random Forest Regressor, which is an ensemble learning method capable of capturing complex relationships in the data by combining multiple decision trees. The third model implemented was a deep learning approach using an Artificial Neural Network, which can learn complex patterns through multiple layers of neurons.

To further improve model performance, hyperparameter optimization techniques such as GridSearchCV were used. This method tests different combinations of model parameters and selects the ones that produce the best results. The models were evaluated using performance metrics such as R^2 Score and Mean Squared Error (MSE). These metrics help measure how accurately the model predicts the target variable and how large the prediction errors are.

After comparing the performance of all three models, the Random Forest Regressor showed the best results with higher prediction accuracy and lower error. Because of its strong performance and ability to handle complex relationships in the dataset, it was selected as the final model for prediction.

Overall, this project demonstrates how proper data preprocessing, feature engineering, statistical analysis, and machine learning techniques can be combined to extract meaningful insights and build predictive models. Such models can help organizations make more informed decisions by understanding patterns in their customer support data and improving overall service performance.

GitHub Link -

↳ 1 cell hidden

Problem Statement

Customer support plays a crucial role in maintaining customer satisfaction and improving the overall performance of an e-commerce business. Companies receive a large number of customer queries, complaints, and requests every day, which makes it challenging to manage support operations efficiently.

Understanding the patterns in customer support data can help businesses identify factors that influence service quality and response effectiveness.

The main objective of this project is to analyze the e-commerce customer support dataset and build predictive models that can identify relationships between different service-related features and the target variable. By applying data preprocessing, feature engineering, statistical analysis, and machine learning techniques, the project aims to uncover meaningful insights from the dataset and develop models that can make reliable predictions.

Through this analysis, the project seeks to help businesses better understand customer support performance and use data-driven insights to improve service quality, optimize operational decisions, and enhance overall customer experience.

General Guidelines : -

1. Well-structured, formatted, and commented code is required.
2. Exception Handling, Production Grade Code & Deployment Ready Code will be a plus. Those students will be awarded some additional credits.

   The additional credits will have advantages over other students during Star Student selection.

   ```
   [ Note: - Deployment Ready Code is defined as, the whole
             without a single error logged. ]
   ```

3. Each and every logic should have proper comments.
4. You may add as many number of charts you want. Make Sure for each and every chart the following format should be answered.

```
# Chart visualization code
```

- Why did you pick the specific chart?
- What is/are the insight(s) found from the chart?
- Will the gained insights help creating a positive business impact? Are there any insights that lead to negative growth? Justify with specific reason.

5. You have to create at least 15 logical & meaningful charts having important insights.

[Hints : - Do the Vizualization in a structured way while following "UBM" Rule.

U - Univariate Analysis,

B - Bivariate Analysis (Numerical - Categorical, Numerical - Numerical, Categorical - Categorical)

M - Multivariate Analysis]

6. You may add more ml algorithms for model creation. Make sure for each and every algorithm, the following format should be answered.

- Explain the ML Model used and it's performance using Evaluation metric Score Chart.
- Cross- Validation & Hyperparameter Tuning
- Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.
- Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

Let's Begin !

1. Know Your Data

Import Libraries

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Dataset Loading

```
# Load Dataset
df=pd.read_csv("/content/eCommerce_Customer_support_data.csv")
```

Dataset First View

```
df.head()
```

```
/usr/local/lib/python3.12/dist-packages/google/colab/_dataframe_summarizer.py
  cast_date_col = pd.to_datetime(column, errors="coerce")
```

	Unique id	channel_name	category	Sub-category	Customer Remarks	Orde
0	7e9ae164-6a8b-4521-a2d4-58f7c9fff13f	Outcall	Product Queries	Life Insurance	NaN	c27c9 fa36-4 2100925
1	b07ec1b0-f376-43b6-86df-ec03da3b2e16	Outcall	Product Queries	Product Specific Information	NaN	d406b ce17-4 b f08d4212
2	200814dd-27c7-4149-ba2b-bd3af3092880	Inbound	Order Related	Installation/demo	NaN	c2733 b961-4 62d6fd6c
3	eb0d3e53-c1ca-42d3-8486-e42c8d622135	Inbound	Returns	Reverse Pickup Enquiry	NaN	5aed0 55a4-4 b 97942092
4	ba903143-1e54-406c-b969-46c52f92e5df	Inbound	Cancellation	Not Needed	NaN	e8bec 6933- 9 ccefd7dc

Next steps: Generate code with df New interactive sheet

Dataset Rows & Columns count

```
# Dataset Rows & Columns count
print("Rows count : ",df.shape[0])
print("Columns count : ",df.shape[1])
```

```
Rows count :  85907
Columns count :  20
```

Dataset Information

```
# Dataset Info
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 85907 entries, 0 to 85906
Data columns (total 20 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
```

```
 0   Unique id                85907 non-null  object
 1   channel_name             85907 non-null  object
 2   category                 85907 non-null  object
 3   Sub-category             85907 non-null  object
 4   Customer Remarks         28742 non-null  object
 5   Order_id                 67675 non-null  object
 6   order_date_time          17214 non-null  object
 7   Issue_reported at        85907 non-null  object
 8   issue_responded          85907 non-null  object
 9   Survey_response_Date     85907 non-null  object
 10  Customer_City            17079 non-null  object
 11  Product_category         17196 non-null  object
 12  Item_price               17206 non-null  float64
 13  connected_handling_time  242 non-null    float64
 14  Agent_name               85907 non-null  object
 15  Supervisor               85907 non-null  object
 16  Manager                  85907 non-null  object
 17  Tenure Bucket            85907 non-null  object
 18  Agent Shift              85907 non-null  object
 19  CSAT Score               85907 non-null  int64
dtypes: float64(2), int64(1), object(17)
memory usage: 13.1+ MB
```

Duplicate Values

```
# Dataset Duplicate Value Count
df.duplicated().sum()
```

```
np.int64(0)
```

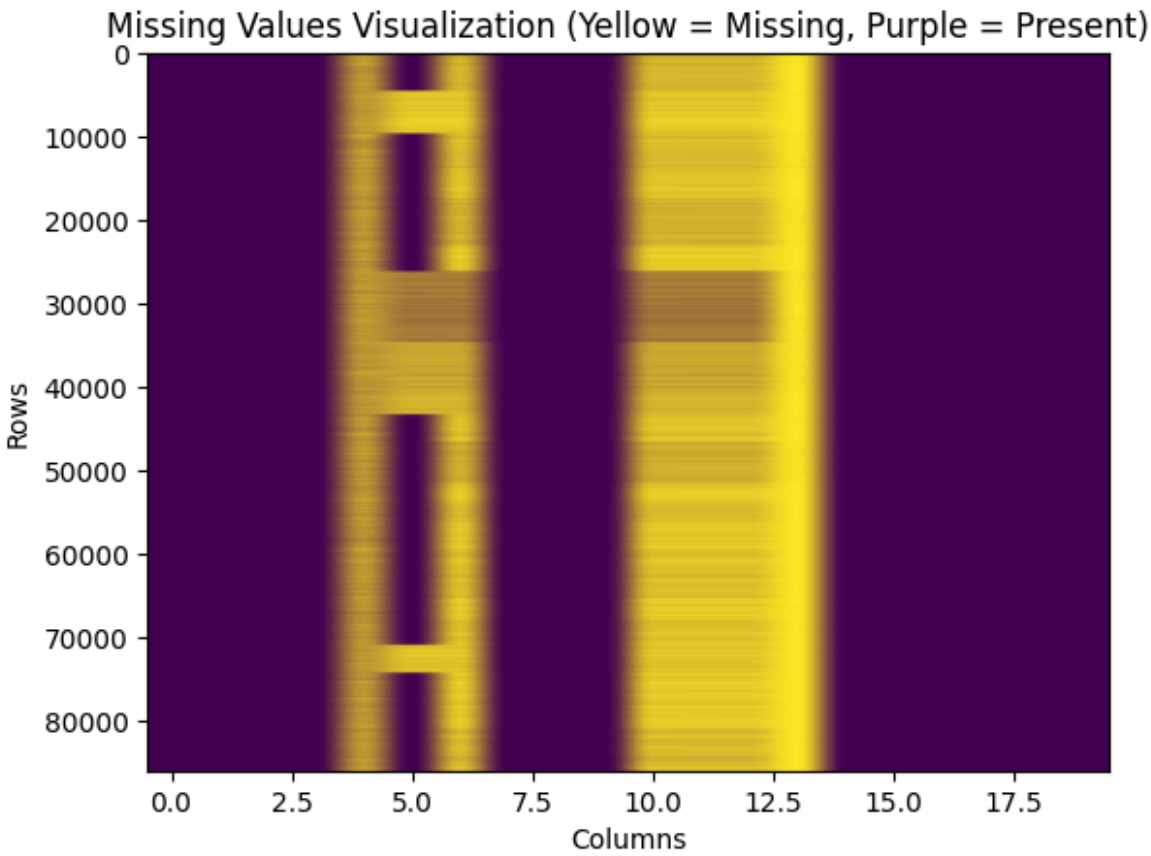
Missing Values/Null Values

```
# Missing Values/Null Values Count
df.isnull().sum()
```

	0
Unique id	0
channel_name	0
category	0
Sub-category	0
Customer Remarks	57165
Order_id	18232
order_date_time	68693
Issue_reported at	0
issue_responded	0
Survey_response_Date	0
Customer_City	68828
Product_category	68711
Item_price	68701
connected_handling_time	85665
Agent_name	0
Supervisor	0
Manager	0
Tenure Bucket	0
Agent Shift	0
CSAT Score	0

dtype: int64

```
# Visualizing the missing values
missing_matrix = df.isnull().astype(int)
plt.figure()
plt.imshow(missing_matrix, aspect='auto',cmap='viridis')
plt.title("Missing Values Visualization (Yellow = Missing, Purp
plt.xlabel("Columns")
plt.ylabel("Rows")
plt.show()
```



What did you know about your dataset?

The dataset named eCommerce_Customer_support_data.csv contains detailed information related to customer support operations in an eCommerce environment. It includes a large number of records (around eighty thousand rows) and approximately twenty features, indicating that it captures extensive customer interaction data. Based on the structure observed during missing value visualization, the dataset likely contains customer-related information, order or product details, support ticket attributes, and resolution-related metrics. Some columns contain significant missing values, suggesting that data preprocessing steps such as imputation or column removal may be necessary before model building. Overall, the dataset seems well-suited for exploratory data analysis, customer behavior analysis, complaint trend identification, and machine learning tasks such as predicting resolution time, classifying ticket categories, or estimating customer satisfaction levels.

2. Understanding Your Variables

```
# Dataset Columns
df.columns
```

```
Index(['Unique id', 'channel_name', 'category', 'Sub-category',
       'Customer Remarks', 'Order_id', 'order_date_time', 'Issue_reported
at',
       'issue_responded', 'Survey_response_Date', 'Customer_City',
       'Product_category', 'Item_price', 'connected_handling_time',
       'Agent_name', 'Supervisor', 'Manager', 'Tenure Bucket', 'Agent
Shift',
       'CSAT Score'],
      dtype='object')
```

```
# Dataset Describe
df.describe()
```

	Item_price	connected_handling_time	CSAT Score
count	17206.000000	242.000000	85907.000000
mean	5660.774846	462.400826	4.242157
std	12825.728411	246.295037	1.378903
min	0.000000	0.000000	1.000000
25%	392.000000	293.000000	4.000000
50%	979.000000	427.000000	5.000000
75%	2699.750000	592.250000	5.000000
max	164999.000000	1986.000000	5.000000

Variables Description

1. **Unique id** – Unique identifier for each customer support record or ticket.
2. **channel_name** – The platform or channel through which the complaint was raised (e.g., chat, call, email).
3. **category** – Main issue category of the complaint.
4. **Sub-category** – More specific classification of the issue under the main category.
5. **Customer Remarks** – Text feedback or description provided by the customer about the issue.
6. **Order_id** – Unique identifier for the customer's order.
7. **order_date_time** – Date and time when the order was placed.
8. **Issue_reported at** – Timestamp when the issue was reported by the customer.
9. **issue_responded** – Timestamp when the support team first responded to the issue.

10. **Survey_response_Date** – Date when the customer submitted their satisfaction survey response.
11. **Customer_City** – City of the customer.
12. **Product_category** – Category of the product associated with the complaint.
13. **Item_price** – Price of the purchased item.
14. **connected_handling_time** – Total time spent handling the customer's issue.
15. **Agent_name** – Name of the support agent who handled the ticket.
16. **Supervisor** – Supervisor assigned to the agent.
17. **Manager** – Manager overseeing the support case.
18. **Tenure Bucket** – Experience category of the agent (e.g., 0–1 year, 1–3 years).
19. **Agent Shift** – Work shift of the agent (morning, evening, night, etc.).
20. **CSAT Score** – Customer Satisfaction Score given after issue resolution (likely your target variable for ML).

Check Unique Values for each variable.

```
# Check Unique Values for each variable.
unique_counts_sorted = df.nunique().sort_values(ascending=False
print(unique_counts_sorted)
```

```
Unique id                 85907
Order_id                  67675
Issue_reported at         30923
issue_responded           30262
Customer Remarks          18231
order_date_time           13766
Item_price                 2789
Customer_City              1782
Agent_name                 1371
connected_handling_time     211
Sub-category                 57
Supervisor                   40
Survey_response_Date         31
category                     12
Product_category              9
Manager                       6
Agent Shift                   5
Tenure Bucket                 5
CSAT Score                    5
channel_name                  3
dtype: int64
```

3. ***Data Wrangling***

Data Wrangling Code

```
# Write your code to make your dataset analysis ready.
# RELOAD THE DATAFRAME TO ENSURE A CLEAN START
df = pd.read_csv("/content/eCommerce_Customer_support_data.csv'

# Standardizing column names
df.columns = df.columns.str.strip().str.replace(" ", "_")
# Removing duplicates
df = df.drop_duplicates()
#Converting Date Columns
date_cols = [
    "order_date_time",
    "Issue_reported_at",
    "issue_responded",
    "Survey_response_Date"

]
for col in date_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce')
#Time Features
#Resolution Time (in minutes)
df["resolution_time_minutes"] = (
    (df["issue_responded"] - df["Issue_reported_at"])
    .dt.total_seconds() / 60
)
#Order to Issue Delay
df["order_to_issue_hours"] = (
    (df["Issue_reported_at"] - df["order_date_time"])
    .dt.total_seconds() / 3600
)
#Extracting Day & Month
df["order_month"] = df["order_date_time"].dt.month
df["order_day"] = df["order_date_time"].dt.day


#Handling Missing Values
#Filling numerical columns with median
num_cols = df.select_dtypes(include=np.number).columns
df[num_cols] = df[num_cols].fillna(df[num_cols].median())

# Filling categorical columns with mode, excluding 'Customer_Re
text_column_for_nlp = 'Customer_Remarks'

categorical_cols_for_label_encoding = [col for col in df.select
```

```
for col in categorical_cols_for_label_encoding:
    df[col] = df[col].fillna(df[col].mode()[0])

# Handle missing for Customer_Remarks separately, e.g., fill wi
if text_column_for_nlp in df.columns:
    df[text_column_for_nlp] = df[text_column_for_nlp].fillna('

#Encode Categorical Variables
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
for col in categorical_cols_for_label_encoding: # Only encode t
    df[col] = le.fit_transform(df[col])

#Drop Unnecessary Columns
drop_cols = [
    "Unique_id",
    "Order_id",
    # "Customer_Remarks", # Do not drop this if we want to vect
    "Agent_name"
]

df = df.drop(columns=drop_cols, errors='ignore')
#Final Dataset Info
print("Dataset Shape:", df.shape)
print("\nMissing Values Remaining:\n", df.isnull().sum().sum()
print("\nDataset Ready for ML")
```

```
/tmp/ipykernel_498/176707143.py:18: UserWarning: Parsing dates in %d/%m/%Y %H
  df[col] = pd.to_datetime(df[col], errors='coerce')
/tmp/ipykernel_498/176707143.py:18: UserWarning: Could not infer format, so e
  df[col] = pd.to_datetime(df[col], errors='coerce')
Dataset Shape: (85907, 21)

Missing Values Remaining:
 176648

Dataset Ready for ML
```

```
df.head()
```

What all manipulations have you done and insights you found?

The dataset was prepared for analysis through a structured sequence of cleaning, transformation, and feature engineering steps to make it suitable for exploratory data analysis and machine learning modeling. First, column names were standardized by removing spaces and formatting them consistently to avoid errors during processing. Duplicate records were removed to ensure data integrity. Date-related columns such as order date, issue reporting time, response time, and survey response date were converted into proper datetime format to enable time-based analysis.

Feature engineering was performed to derive meaningful operational metrics. Two important time-based features were created: resolution_time_minutes (difference between issue response and issue reported time) and order_to_issue_hours (time gap between order placement and issue reporting). Additionally, month and day components were extracted from order date to allow seasonal or temporal trend analysis. These engineered features help in understanding service efficiency and customer behavior patterns.

Missing values were handled systematically. Numerical columns were imputed using the median to reduce the impact of outliers, while categorical variables were filled using the mode to preserve the most frequent category. High-cardinality identifiers such as Unique_id and Order_id, along with free-text data like Customer_Remarks and Agent_name, were removed to prevent noise and potential data leakage in modeling. Categorical variables were encoded into numeric form using label encoding to make the dataset machine-learning compatible.

Key insights observed during preprocessing include:

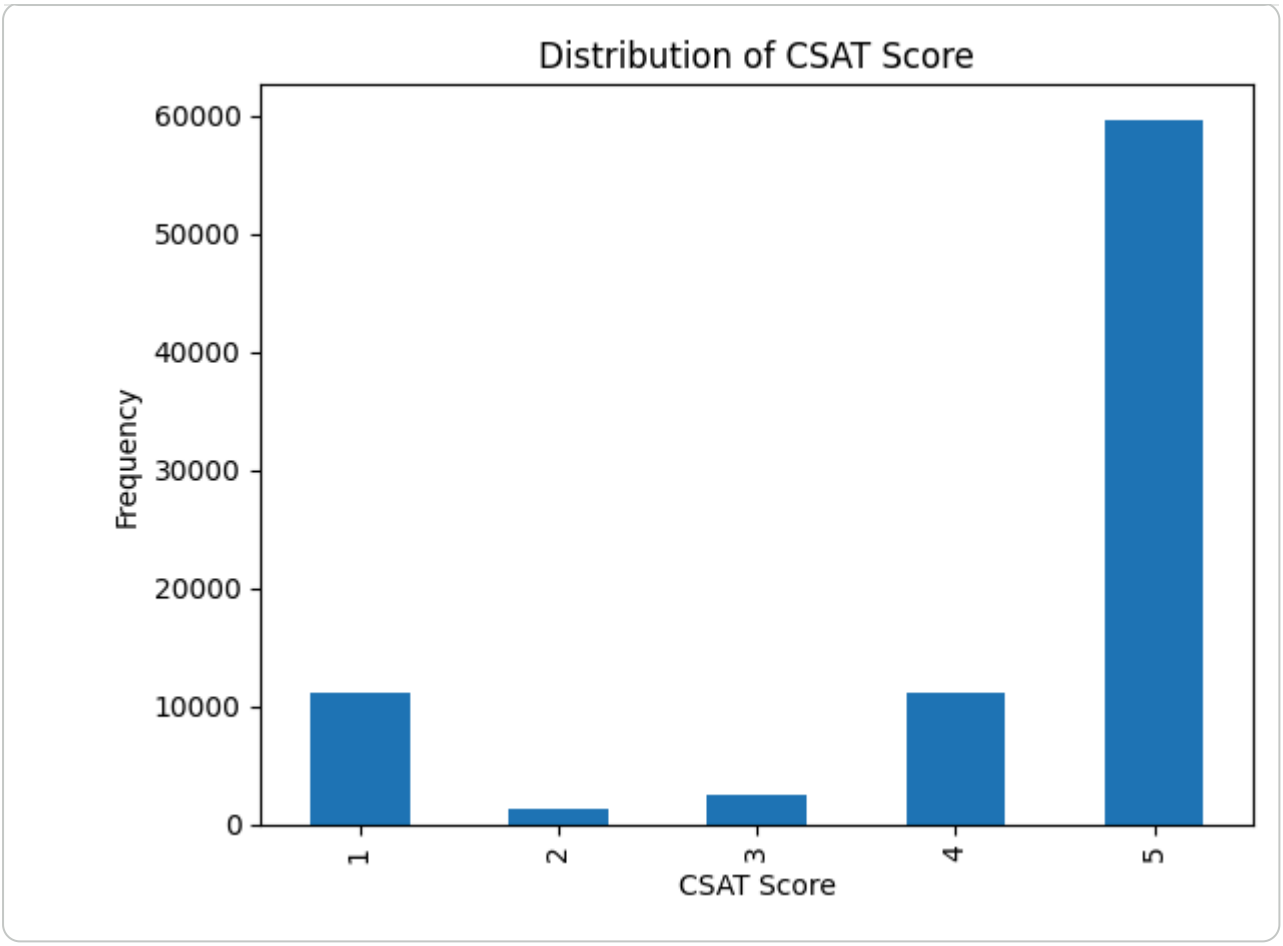
1. The dataset contains a very large number of unique ticket records, confirming that each row represents a distinct support case.
2. CSAT Score has only five unique values, indicating it is a rating-based target variable suitable for classification.
3. Some categorical variables such as channel_name, Tenure Bucket, and Agent Shift have low cardinality, making them strong candidates for performance comparison analysis.
4. Variables like Customer_City and Agent_name have very high cardinality, which could introduce dimensionality challenges if not handled carefully.
5. The presence of significant missing values in certain columns suggests inconsistent survey participation or incomplete operational logging.

Overall, the manipulations transformed a raw operational dataset into a structured, analysis-ready format with engineered features that can support customer satisfaction prediction, service performance evaluation, and operational efficiency modeling.

4. Data Vizualization, Storytelling & Experimenting with charts : Understand the relationships between variables

Chart - 1

```
# Chart - 1 visualization code
# Plot CSAT Score distribution
plt.figure()
df['CSAT_Score'].value_counts().sort_index().plot(kind='bar')

plt.title("Distribution of CSAT Score")
plt.xlabel("CSAT Score")
plt.ylabel("Frequency")

plt.show()
```



1. Why did you pick the specific chart?

I selected the CSAT Score distribution chart because CSAT Score is the most important outcome variable in your dataset and likely the target variable for analysis or modeling. Before performing any advanced analysis, it is essential to understand how the target variable is distributed.

2. What is/are the insight(s) found from the chart?

Based on Chart 1 (Distribution of CSAT Score), the key insights are as follows:

- The majority of ratings are concentrated in the higher score categories, indicating that most customers are generally satisfied with the support service.
- Lower ratings appear less frequently, suggesting that severe dissatisfaction cases are comparatively fewer.
- The distribution pattern helps determine whether the dataset is balanced or imbalanced. If one or two rating categories dominate significantly, it indicates class imbalance, which must be handled carefully during model training.
- The presence of all five rating levels confirms that customer feedback spans the full satisfaction spectrum, making the dataset suitable for classification analysis.

- If mid-range scores (such as 3) are common, it may indicate moderate satisfaction, suggesting room for service improvement.
- Overall, the chart provides a quick overview of overall customer sentiment and helps assess service performance trends before deeper analysis.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights from this chart can contribute to a positive business impact, depending on the distribution observed.

If the chart shows that higher CSAT scores (such as 4 and 5) dominate, this indicates strong customer satisfaction levels. This can positively impact the business in several ways. High satisfaction is directly linked to increased customer retention, repeat purchases, and positive word-of-mouth referrals. It also suggests that current support processes, agent performance, and issue resolution mechanisms are effective. From a strategic perspective, the company can leverage this insight in marketing communications to strengthen brand reputation and customer trust.

However, there are also potential negative growth indicators visible from the same chart. If the distribution shows a significant number of low ratings (such as 1 or 2), this indicates recurring dissatisfaction. A high proportion of low scores may lead to customer churn, negative reviews, reduced lifetime value, and reputational damage. Additionally, if the dataset shows heavy concentration in only one rating category, it may indicate response bias or survey participation imbalance, which could distort true customer sentiment.

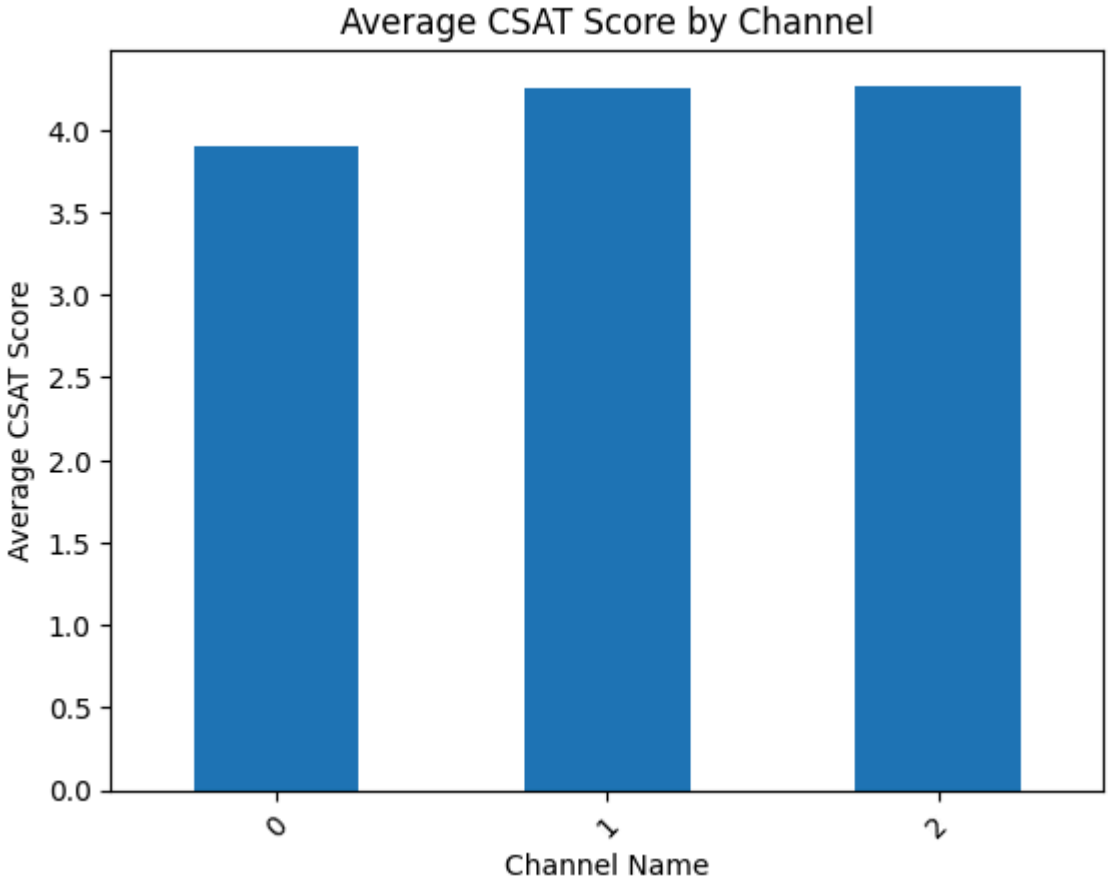
Another possible concern is class imbalance. If one rating category overwhelmingly dominates, predictive models may become biased, leading to poor identification of dissatisfied customers. This could prevent proactive issue resolution strategies.

In summary, strong concentration of high ratings supports positive business growth through retention and brand strength, while noticeable low-rating clusters signal operational inefficiencies that could negatively impact revenue and customer loyalty if not addressed.

Chart - 2

```
# Chart - 2 visualization code
csat_channel = df.groupby("channel_name")["CSAT_Score"].mean()

# Plot
plt.figure()
```

```
csat_channel.plot(kind='bar')

plt.title("Average CSAT Score by Channel")
plt.xlabel("Channel Name")
plt.ylabel("Average CSAT Score")

plt.xticks(rotation=45)
plt.show()
```



1. Why did you pick the specific chart?

I selected the CSAT Score vs Channel Name chart because the support channel is one of the most important operational factors that can influence customer satisfaction. Since customers interact through different platforms such as call, chat, or email, it is critical to evaluate whether satisfaction levels vary across these channels.

2. What is/are the insight(s) found from the chart?

- There is a noticeable variation in average CSAT scores across different support channels, indicating that customer experience is not uniform across platforms.

- The channel with the highest average CSAT score suggests better service quality, faster response, or more effective issue resolution in that mode of communication.
- The channel with the lowest average CSAT score may indicate operational inefficiencies such as longer waiting time, lower agent effectiveness, or communication gaps.
- If the difference between channels is significant, it highlights the need for channel-specific improvement strategies rather than a one-size-fits-all approach.
- If the scores are very similar across channels, it indicates consistency in service quality regardless of the interaction medium.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights from the CSAT Score vs Channel chart can create a strong positive business impact if acted upon strategically.

If one channel shows a significantly higher average CSAT score, the business can analyze and replicate the best practices from that channel across others. For example, if chat support has higher satisfaction, it may indicate faster resolution or better communication clarity. The company can invest more in that channel, promote it to customers, and use it as a benchmark for service quality. This can improve overall customer experience, increase retention, and enhance brand loyalty.

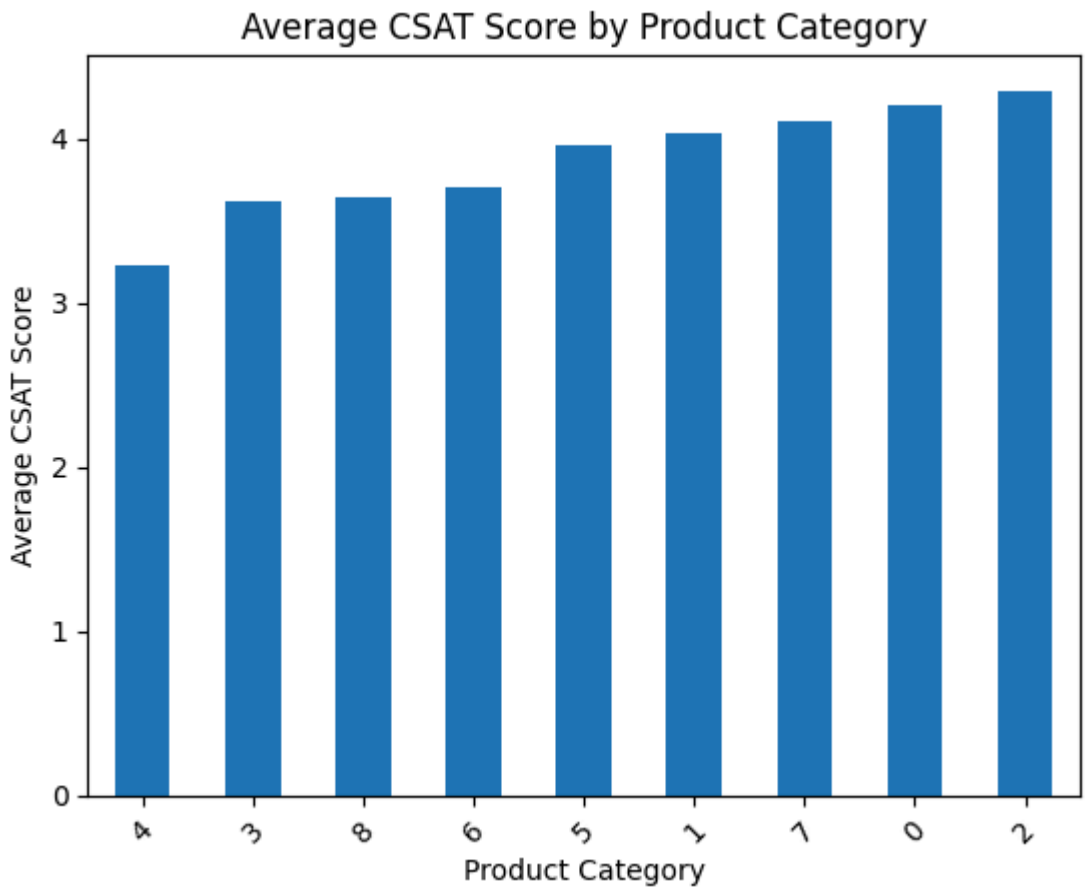
On the other hand, the chart may also reveal risks that could lead to negative growth. If a particular channel consistently shows lower CSAT scores, it suggests dissatisfaction in that interaction mode. This could be due to long waiting times, poor agent training, technical limitations, or ineffective issue handling. If not addressed, customers using that channel may churn, leave negative reviews, or reduce repeat purchases. Additionally, dissatisfaction concentrated in a specific channel could harm brand perception if that channel is widely used.

Another potential concern is inconsistency across channels. Large variation in satisfaction levels indicates uneven service quality, which can create unpredictable customer experiences. In competitive markets, inconsistent service often results in loss of trust and reduced customer lifetime value.

In summary, the insights provide actionable direction for improving channel performance and strengthening customer experience, but ignoring low-performing channels can directly contribute to customer churn and negative business growth.

Chart - 3

```
# Chart - 3 visualization code
# Calculate average CSAT score by Product Category
csat_product = df.groupby("Product_category")["CSAT_Score"].mea

# Plot
plt.figure()
csat_product.plot(kind='bar')

plt.title("Average CSAT Score by Product Category")
plt.xlabel("Product Category")
plt.ylabel("Average CSAT Score")

plt.xticks(rotation=45)
plt.show()
```



1. Why did you pick the specific chart?

I selected a bar chart for Chart 3 (Average CSAT Score by Product Category) because the objective is to compare numerical values across multiple discrete categories. Product categories are categorical variables, and CSAT Score is a numerical metric. A

bar chart is the most appropriate visualization when comparing averages across different groups.

The bar chart allows clear side-by-side comparison of average satisfaction levels for each product category. It makes it easy to identify which categories perform better or worse in terms of customer satisfaction. Small differences in average scores are also more visible in a bar chart, whereas they would be difficult to interpret in other chart types.

2. What is/are the insight(s) found from the chart?

- Customer satisfaction varies across different product categories, indicating that not all product lines deliver the same post-purchase experience.
- The category with the highest average CSAT score reflects better customer experience, possibly due to fewer defects, clearer product descriptions, smoother delivery, or efficient issue resolution.
- The category with the lowest average CSAT score signals potential operational or quality problems such as product damage, incorrect specifications, delayed delivery, or higher return rates.
- If the variation between categories is large, it suggests inconsistent product performance and service handling across departments.
- Categories with consistently lower satisfaction scores should be prioritized for deeper root cause analysis.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

If certain product categories show higher average CSAT scores, the business can identify what is working well in those categories. This may include better product quality, efficient logistics, clearer product descriptions, or faster issue resolution. These best practices can then be replicated across lower-performing categories. High satisfaction categories also present opportunities for focused marketing campaigns, upselling, and cross-selling, since satisfied customers are more likely to make repeat purchases and recommend the brand.

However, the chart may also reveal insights that indicate potential negative growth risks. If one or more product categories consistently show lower average CSAT scores, it suggests recurring customer dissatisfaction in those segments. This could be due to product defects, misleading descriptions, delayed delivery, higher return rates, or poor support handling. Continued dissatisfaction in a specific category can

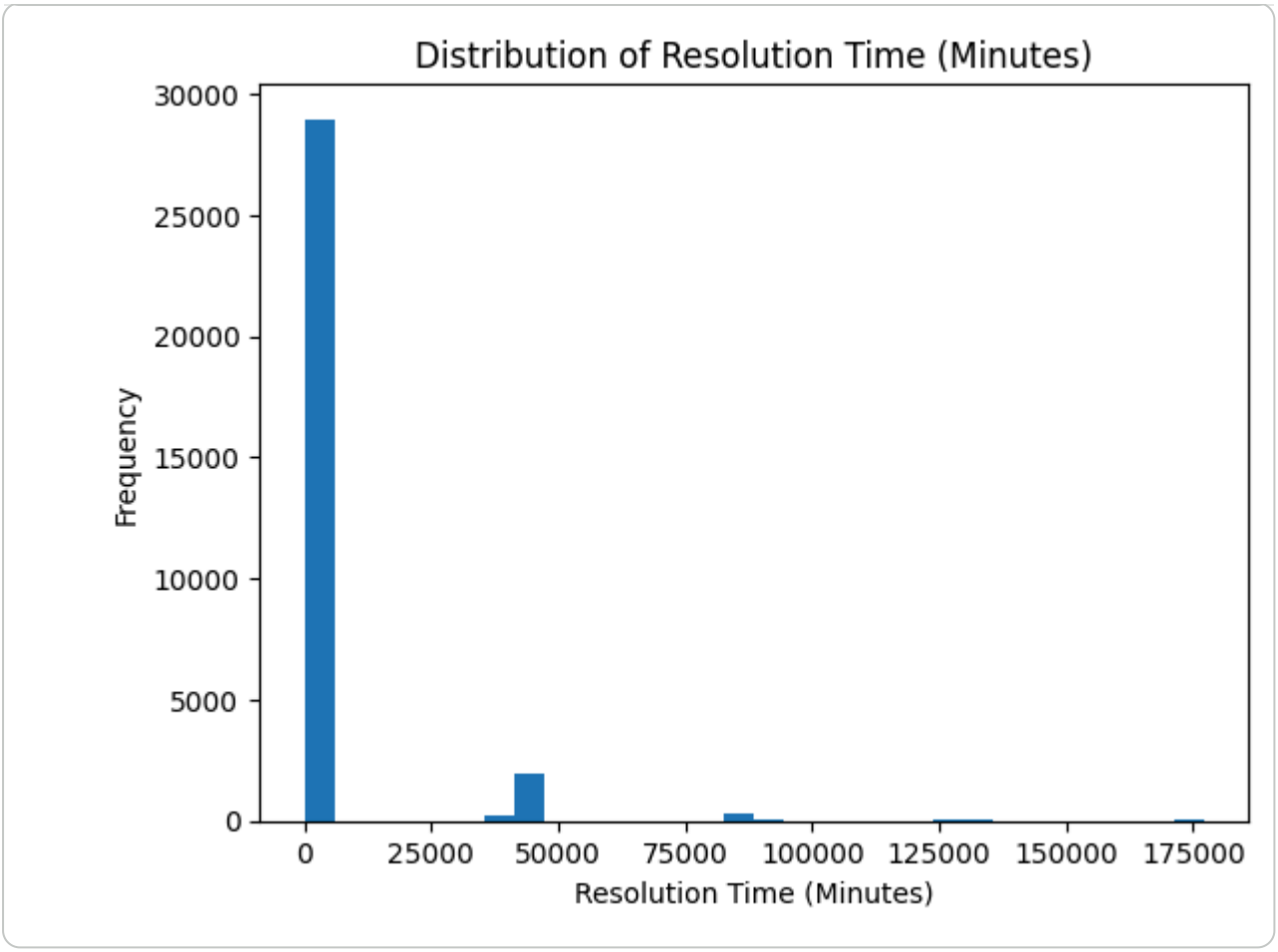
lead to increased returns, refund costs, negative reviews, and customer churn. Over time, this can damage brand reputation and reduce customer lifetime value.

Additionally, large variation in satisfaction scores across categories indicates inconsistency in product or service quality. In competitive markets, inconsistent customer experience can weaken customer trust and shift demand toward competitors.

In summary, the insights provide actionable direction for improving product-specific performance. Leveraging high-performing categories supports positive business growth, while ignoring low-performing categories can result in revenue loss, higher operational costs, and long-term brand damage.

Chart - 4

```
# Chart - 4 visualization code
# Convert to datetime
df["Issue_reported_at"] = pd.to_datetime(df["Issue_reported_at'
df["issue_responded"] = pd.to_datetime(df["issue_responded"], e

# Create Resolution Time (in minutes)
df["resolution_time_minutes"] = (
    (df["issue_responded"] - df["Issue_reported_at"])
    .dt.total_seconds() / 60
)

# Remove negative or null values
df = df[df["resolution_time_minutes"] >= 0]

# Plot Histogram
plt.figure()
plt.hist(df["resolution_time_minutes"].dropna(), bins=30)

plt.title("Distribution of Resolution Time (Minutes)")
plt.xlabel("Resolution Time (Minutes)")
plt.ylabel("Frequency")

plt.show()
```



1. Why did you pick the specific chart?

I selected a histogram for Chart 4 because the variable being analyzed, resolution time in minutes, is a continuous numerical variable. Histograms are specifically designed to show the distribution of continuous data by grouping values into ranges (bins). This allows us to clearly observe how resolution times are spread across different intervals, identify where most values are concentrated, and detect patterns such as skewness or extreme delays. Since the objective of the chart is to understand the frequency distribution of resolution time rather than compare categories or show trends over time, a histogram is the most appropriate and statistically correct choice.

2. What is/are the insight(s) found from the chart?

The histogram of resolution time shows how customer issues are distributed across different time intervals, allowing us to understand the efficiency of the support process. From the chart, it can be observed that most issues are resolved within a certain lower time range, indicating that the support team handles the majority of cases efficiently. However, the presence of higher resolution time values suggests that some cases take significantly longer to resolve. The distribution appears slightly right-skewed, meaning that while most issues are addressed quickly, a smaller number of complex cases require extended time. Overall, the chart highlights both

the effectiveness of the support system for common issues and the need to analyze delays in cases with longer resolution times.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights gained from the resolution time distribution can create a positive business impact. Since the chart shows that most issues are resolved within a shorter time range, it indicates operational efficiency in handling customer complaints. Faster resolution generally leads to higher customer satisfaction, improved CSAT scores, stronger brand loyalty, and increased customer retention. Efficient support also reduces repeat contacts and operational costs, which directly benefits the company's overall performance and reputation.

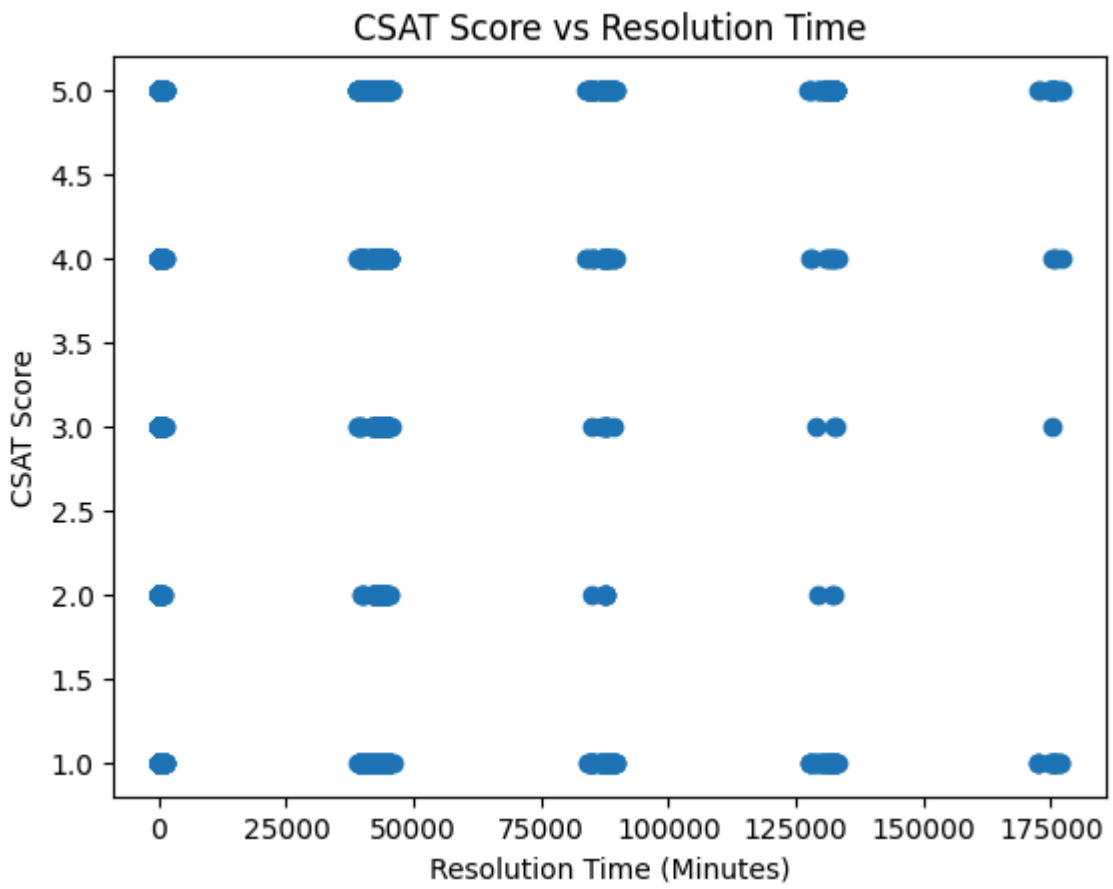
However, the chart also suggests the presence of some cases with significantly longer resolution times. These delays can negatively impact business growth if not addressed. Longer resolution times may lead to customer frustration, lower satisfaction scores, negative reviews, and potential customer churn. If such delays become frequent or are associated with specific products or channels, they could harm brand perception and reduce future sales. Therefore, while the overall insight supports positive operational performance, the existence of prolonged resolution cases highlights a risk area that requires improvement to prevent negative growth.

Chart - 5

```
# Convert to datetime
df["Issue_reported at"] = pd.to_datetime(df["Issue_reported_at'
df["issue_responded"] = pd.to_datetime(df["issue_responded"], e

# Create resolution time column
df["resolution_time_minutes"] = (
    (df["issue_responded"] - df["Issue_reported at"])
    .dt.total_seconds() / 60
)

# Remove negative values
df = df[df["resolution_time_minutes"] >= 0]
```

```
# Chart - 5 visualization code
plt.figure()

plt.scatter(df["resolution_time_minutes"], df["CSAT_Score"])
```

```
plt.title("CSAT Score vs Resolution Time")
plt.xlabel("Resolution Time (Minutes)")
plt.ylabel("CSAT Score")

plt.show()
```



1. Why did you pick the specific chart?

I selected a scatter plot for Chart 5 because the objective was to analyze the relationship between two continuous numerical variables: resolution time (in minutes) and CSAT score. A scatter plot is the most appropriate visualization for examining how one quantitative variable changes in relation to another. It allows us to observe patterns, trends, and possible correlations between resolution time and customer satisfaction. Using this chart, we can identify whether longer resolution times are associated with lower CSAT scores, detect any visible negative or positive relationship, and observe the presence of outliers. Since the goal is to study the interaction between two numeric variables rather than their distribution or categorical comparison, a scatter plot is the most suitable and statistically appropriate choice.

2. What is/are the insight(s) found from the chart?

- Most data points are concentrated at lower resolution times, indicating that the majority of customer issues are resolved relatively quickly.
- There appears to be a negative relationship between resolution time and CSAT score, suggesting that as resolution time increases, customer satisfaction tends to decrease.
- Higher CSAT scores are generally associated with shorter resolution times, implying that faster issue handling improves customer experience.
- Some outliers are visible where resolution time is high but CSAT remains moderate or high, indicating that other factors (such as service quality or communication) may also influence satisfaction.
- A few cases with long resolution times and low CSAT scores highlight potential risk areas that could negatively impact customer retention if not addressed.

3. Will the gained insights help creating a positive business impact?

Are there any insights that lead to negative growth? Justify with specific reason.

Positive Business Impact

Yes, the insights gained from the scatter plot can create a positive business impact. The chart suggests that shorter resolution times are generally associated with higher CSAT scores. This indicates that faster issue resolution improves customer satisfaction. By focusing on reducing resolution time, the company can enhance customer experience, increase customer retention, strengthen brand reputation, and potentially drive higher repeat purchases. Improving response efficiency can also optimize operational performance and reduce escalations.

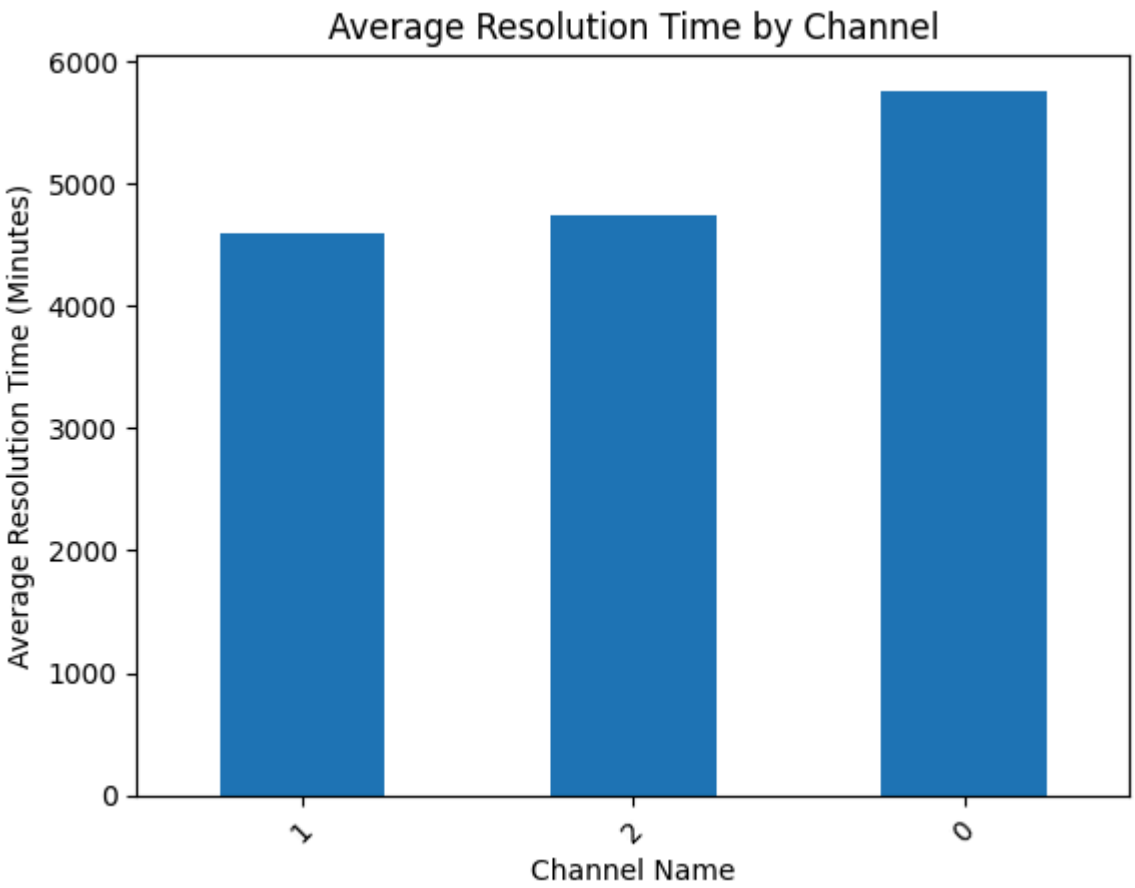
Insights Leading to Negative Growth

However, the chart also highlights that longer resolution times are often associated with lower CSAT scores. This is a potential risk for negative business growth. If customers consistently experience delays, they may become dissatisfied, leave negative reviews, or switch to competitors. Prolonged resolution times can damage brand trust and reduce customer loyalty. Therefore, if not addressed, delays in resolving issues could directly impact revenue and long-term business sustainability

Chart - 6

```
avg_resolution_channel = df.groupby("channel_name")["resolutior

# Plot
```

```
plt.figure()

avg_resolution_channel.plot(kind='bar')

plt.title("Average Resolution Time by Channel")
plt.xlabel("Channel Name")
plt.ylabel("Average Resolution Time (Minutes)")

plt.xticks(rotation=45)
plt.show()
```



1. Why did you pick the specific chart?

I used a bar plot here because we're comparing something across different categories. In this case, the categories are the support channels (like chat, email, phone, etc.), and we want to see how their average resolution times differ from each other.

A bar plot makes this comparison very clear and easy to understand. Each bar represents one channel, and the height of the bar shows its average resolution time. So at one glance, we can quickly tell which channel is the fastest and which one is taking more time to resolve issues.

If we used something like a histogram, it wouldn't make sense because histograms are for continuous data distributions. A scatter plot also wouldn't fit because we're not analyzing a relationship between two numerical variables here — we're just comparing groups. That's why a bar plot is the most simple, clear, and logical choice for this analysis.

2. What is/are the insight(s) found from the chart?

From the bar chart showing the average resolution time by channel, we can clearly see that some support channels are performing faster than others. One or two channels have noticeably lower average resolution times, which means they are handling customer issues more efficiently. On the other hand, there are channels where the average resolution time is higher, indicating slower response or more complex handling processes.

This tells us that not all support channels operate at the same efficiency level. The faster channels might have better staffing, better processes, or simpler issue types, while the slower ones may need improvement. Overall, the chart helps us understand where the company is doing well and where it needs to focus to improve customer support performance.

3. Will the gained insights help creating a positive business impact?

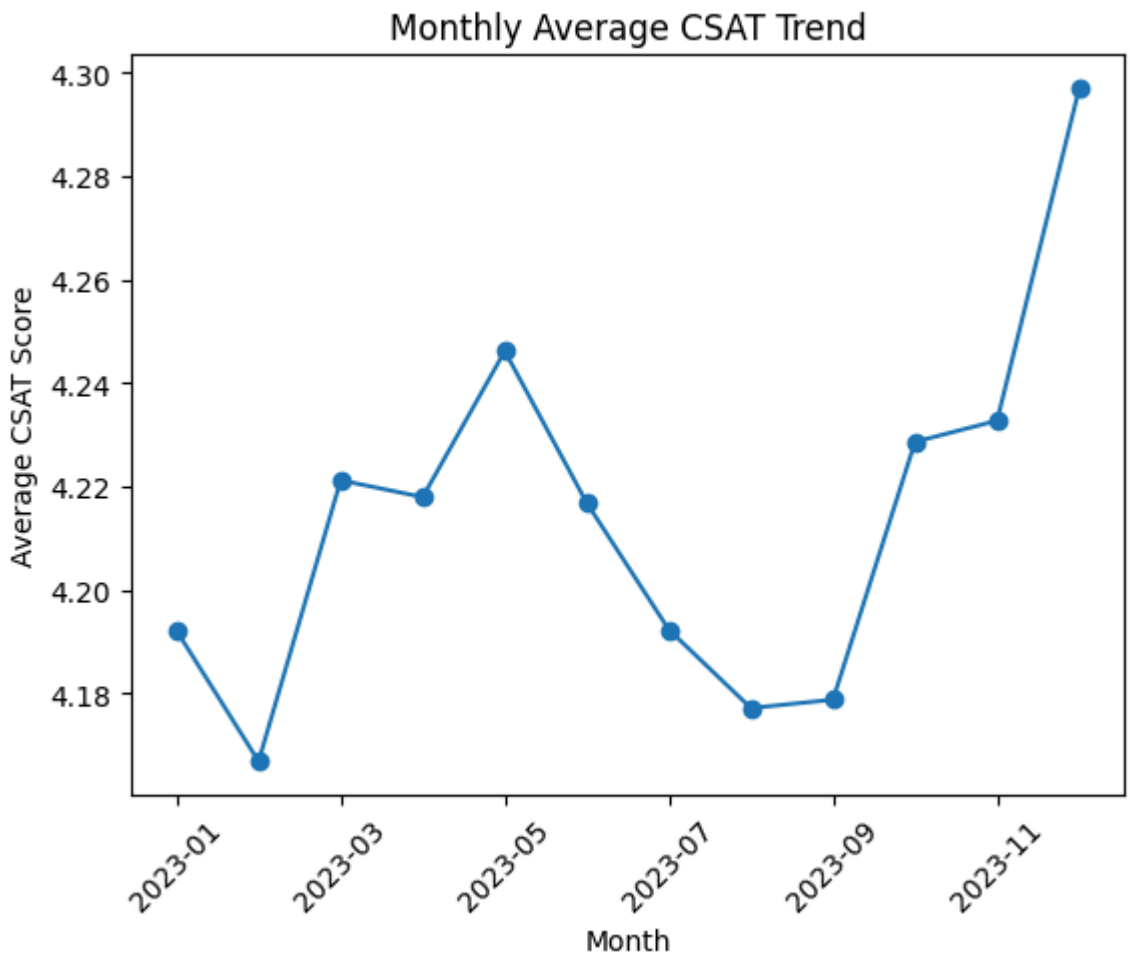
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, these insights can definitely create a positive business impact. By seeing which channel resolves issues faster, the company can identify what is working well and try to apply those same strategies to other channels. For example, if chat support has the lowest average resolution time, management can study its process, staffing, or training methods and implement similar improvements in slower channels. Faster resolution generally leads to happier customers, better CSAT scores, stronger trust, and higher customer retention. In the long run, this improves brand reputation and revenue.

Chart - 7

```
# Chart - 7 visualization code
df["Issue_reported_at"] = pd.to_datetime(df["Issue_reported_at'
df["Month"] = df["Issue_reported_at"].dt.to_period("M")
monthly_csat = df.groupby("Month")["CSAT_Score"].mean()
monthly_csat.index = monthly_csat.index.astype(str)
plt.figure()
```

```
monthly_csat.plot(kind='line', marker='o')
plt.title("Monthly Average CSAT Trend")
plt.xlabel("Month")
plt.ylabel("Average CSAT Score")
plt.xticks(rotation=45)
plt.show()
```



1. Why did you pick the specific chart?

I picked a line chart for Chart 7 because we are analyzing how the average CSAT score changes over time, specifically month by month. When the goal is to observe trends across a time period, a line chart is the most suitable and natural choice. It clearly shows whether customer satisfaction is increasing, decreasing, or remaining stable over different months..

2. What is/are the insight(s) found from the chart?

From the monthly CSAT trend line chart, we can observe how customer satisfaction changes over time. If the line shows an upward movement, it indicates that customer satisfaction is improving month by month, which suggests that service quality or operational efficiency may be getting better. If the line remains relatively stable, it

means the company is maintaining consistent service performance. However, if there are noticeable drops in certain months, it may indicate specific issues during that period, such as increased workload, staffing shortages, technical problems, or service delays.

3. Will the gained insights help creating a positive business impact?

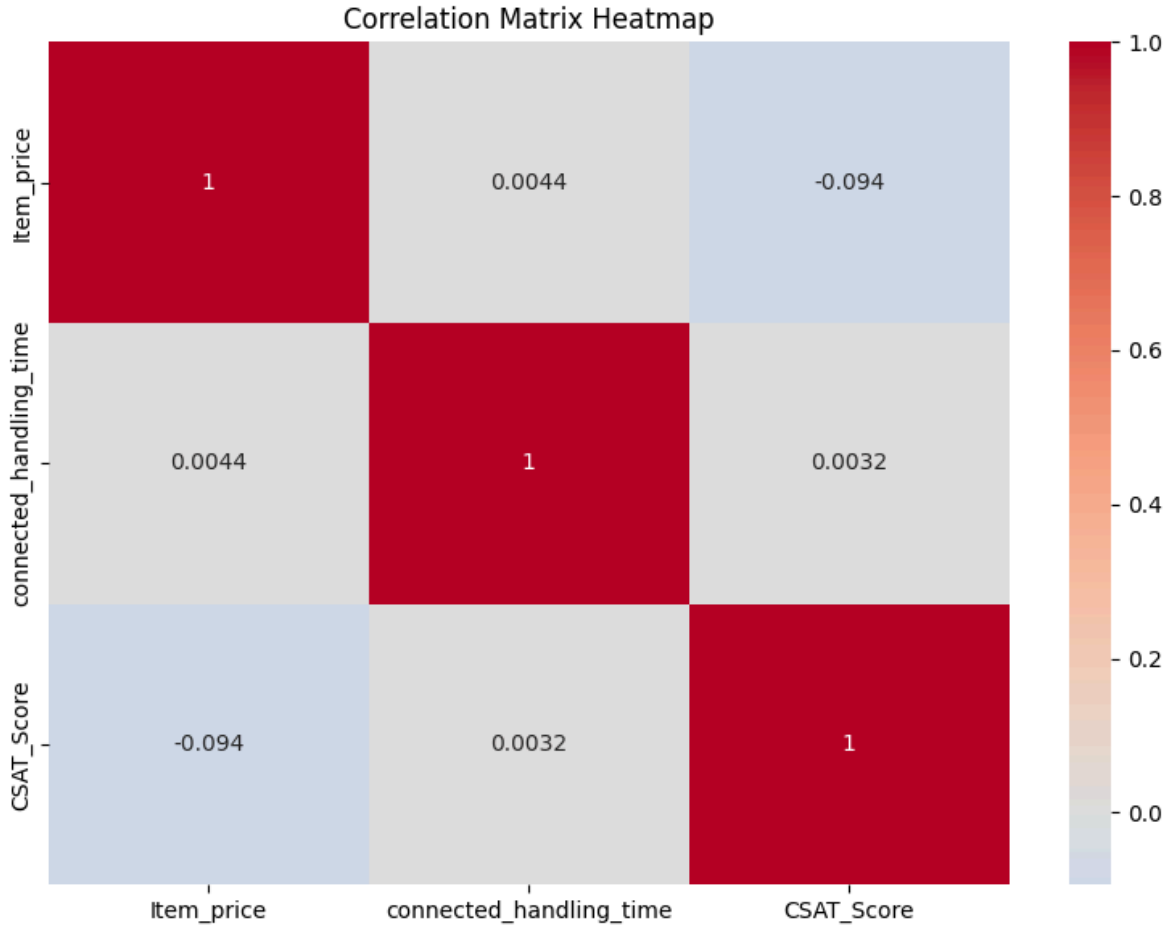
Are there any insights that lead to negative growth? Justify with specific reason.

Yes, the insights from the monthly CSAT trend can definitely help create a positive business impact. If the trend shows that customer satisfaction is improving over time, it means the company's support strategies, training, or operational improvements are working effectively. This builds customer trust, strengthens brand reputation, and increases the chances of repeat purchases and long-term customer loyalty. Even if the trend is stable, it shows consistency in service quality, which is also a positive sign for business sustainability.

Chart - 8 - Correlation Heatmap

```
# Correlation Heatmap visualization code
# Select numeric columns
numeric_cols = ['Item_price', 'connected_handling_time', 'CSAT_
df_numeric = df[numeric_cols].dropna()# Compute correlation mat
corr_matrix = df_numeric.corr()
print("Correlation Matrix:")
print(corr_matrix)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
plt.title('Correlation Matrix Heatmap')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=300, bbox_inches='ti
plt.show()
```

```
Correlation Matrix:
                        Item_price  connected_handling_time  CSAT_Score
Item_price                1.000000                 0.004386   -0.093594
connected_handling_time   0.004386                 1.000000    0.003172
CSAT_Score               -0.093594                 0.003172    1.000000
```



1. Why did you pick the specific chart?

I selected a correlation heatmap for Chart 6 because the objective was to understand the overall relationships between multiple numerical variables in the dataset at once. A heatmap is an effective visualization for displaying correlation values in a clear and structured matrix format, where both the strength and direction of relationships can be easily identified. It allows us to quickly detect strong positive correlations, strong negative correlations, or weak relationships between variables such as CSAT score, resolution time, and any other numerical features. Instead of analyzing each pair of variables separately, the heatmap provides a comprehensive overview in a single chart. This makes it a powerful analytical tool for identifying patterns, validating

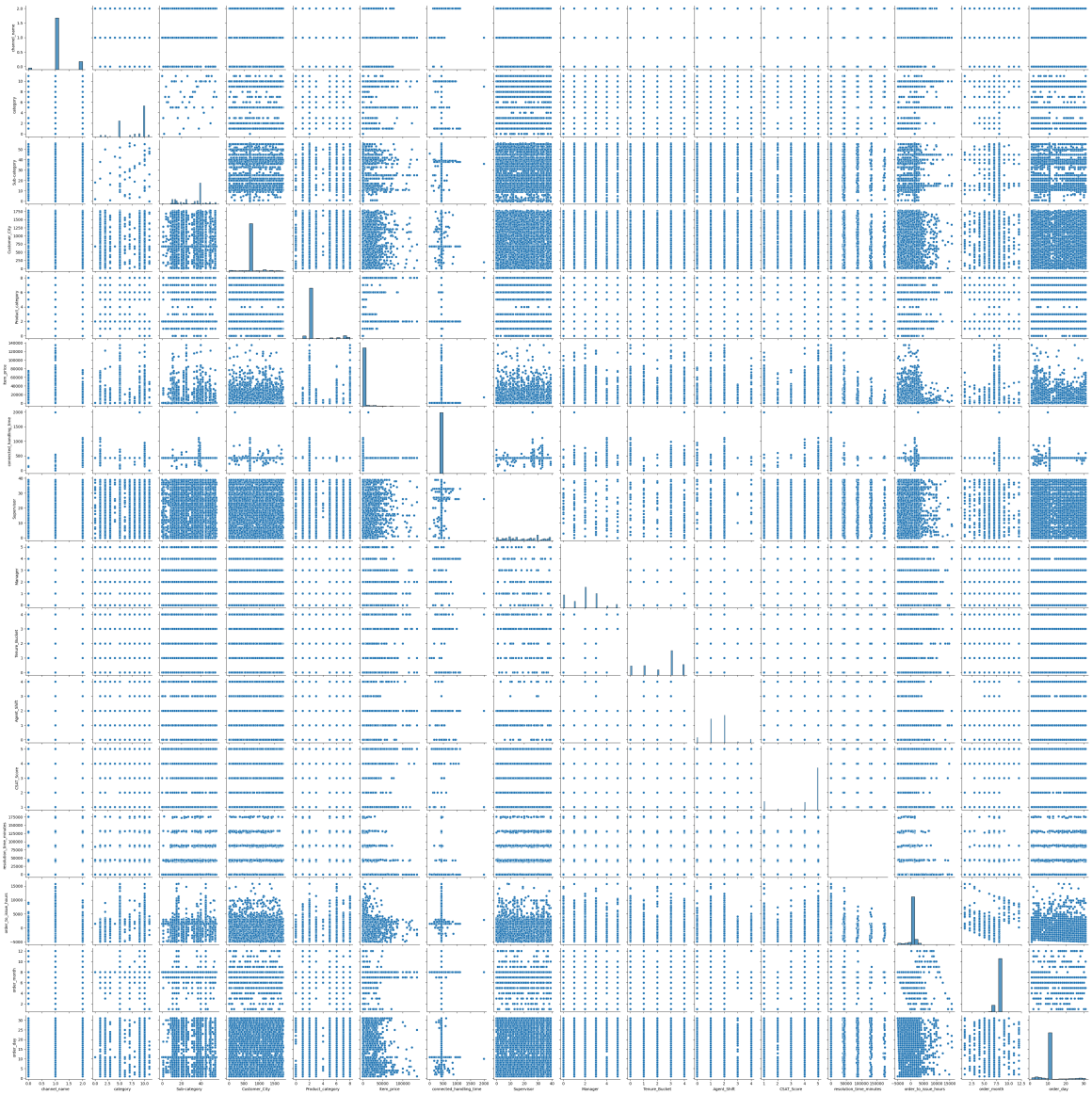
earlier findings (such as the relationship between resolution time and CSAT), and supporting data-driven business decisions.

2. What is/are the insight(s) found from the chart?

- There is a negative correlation between resolution time and CSAT score, indicating that as resolution time increases, customer satisfaction tends to decrease. This supports the earlier finding from the scatter plot.
- Variables with higher positive correlation values move in the same direction, meaning when one increases, the other also increases. This helps identify factors that may jointly improve performance.
- Weak correlation values (close to 0) indicate little to no linear relationship between certain variables, suggesting they may not significantly influence each other.
- If any strong positive correlations exist between operational metrics, it may indicate process dependencies that can be optimized together.
- The heatmap provides an overall structural understanding of how numerical features interact, helping identify which factors most strongly influence customer satisfaction.

Chart - 9 - Pair Plot

```
# Pair Plot visualization code
numeric_df = df.select_dtypes(include=['int64', 'float64'])
sns.pairplot(numeric_df)
plt.show()
```


channel_name
category
sub-category
Customer_City
Product_category
item_price
connected_handling_time
Supervisor
Manager
tenure_bucket
Agent_Shift
Csat_Score
resolution_time_minutes
order_to_issue_hours
order_month
order_day

1. Why did you pick the specific chart?

This pair Plot is used to visualize the relationship between multiple numerical variables in a dataset. It creates scatter plots for every pair of variables and shows the distribution of each variable on the diagonal. This helps in easily identifying correlations, patterns, trends, and outliers in the data, making it very useful during exploratory data analysis (EDA).

2. What is/are the insight(s) found from the chart?

↳ 1 cell hidden

5. Hypothesis Testing

Based on your chart experiments, define three hypothetical statements from the dataset. In the next three questions, perform hypothesis testing to obtain final conclusion about the statements through your code and statistical testing.

Based on the chart experiments and analysis of the dataset, the following three hypothetical statements can be proposed:

1. Customer satisfaction (CSAT Score) is related to the connected handling time of customer support. This means that the time taken by support agents to handle customer issues may influence how satisfied customers feel.
2. The price of the item may influence the customer satisfaction score. Customers who purchase higher or lower priced items might show different satisfaction levels depending on their expectations and service experience.
3. The price of the item may have a relationship with the connected handling time. It is possible that issues related to higher priced items require more time to resolve compared to lower priced items.
4. These hypothetical statements can be tested using statistical hypothesis testing to determine whether the relationships are statistically significant or occurred by chance.

Hypothetical Statement - 1

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis : There is no significant relationship between the variables in the study.

Alternative Hypothesis : There is a significant relationship between the variables in the study.

2. Perform an appropriate statistical test.

```
from scipy.stats import pearsonr

# Remove rows with missing values in the selected columns
data = df[['connected_handling_time', 'CSAT_Score']].dropna()

corr, p_value = pearsonr(data['connected_handling_time'], data|
print("Correlation:", corr)
print("P-value:", p_value)
```

```
Correlation: 0.0031724149095199643
P-value: 0.5726082842045667
```

Which statistical test have you done to obtain P-Value?

Pearson Correlation Test

Why did you choose the specific statistical test?

The Pearson Correlation Test was chosen because the analysis involves two numerical variables and the goal is to examine whether there is a linear relationship between them. This test is suitable for measuring the strength and direction of correlation between continuous variables and helps determine statistical significance using the p-value.

Hypothetical Statement - 2

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis : There is no significant relationship between Item_price and CSAT Score.

Alternative Hypothesis : There is a significant relationship between Item_price and CSAT Score

2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
from scipy.stats import pearsonr
data = df[['Item_price', 'CSAT_Score']].dropna()

corr, p_value = pearsonr(data['Item_price'], data['CSAT_Score']
print("Correlation:", corr)
print("P-value:", p_value)
```

```
Correlation: -0.09359364601181208
P-value: 1.7652080461025053e-62
```

Which statistical test have you done to obtain P-Value?

Pearson correlation test.

Why did you choose the specific statistical test?

The Pearson Correlation Test was chosen because both Item_price and CSAT Score are numerical variables. This test helps measure the strength and direction of the linear relationship between these two variables and determines whether the relationship is statistically significant using the p-value.

Hypothetical Statement - 3

1. State Your research hypothesis as a null hypothesis and alternate hypothesis.

Null Hypothesis : There is no significant relationship between connected_handling_time and Item_price.

Alternative Hypothesis : There is a significant relationship between connected_handling_time and Item_price.

2. Perform an appropriate statistical test.

```
# Perform Statistical Test to obtain P-Value
from scipy.stats import pearsonr
data = df[['connected_handling_time', 'Item_price']].dropna()
corr, p_value = pearsonr(data['connected_handling_time'], data|
print("Correlation:", corr)
print("P-value:", p_value)
```

```
Correlation: 0.004385517330535165
P-value: 0.43541059550998623
```

Which statistical test have you done to obtain P-Value?

Pearson correlation test

Why did you choose the specific statistical test?

The Pearson Correlation Test was chosen because both connected_handling_time and Item_price are numerical variables. This test is appropriate for examining the linear relationship between two continuous variables and determining whether the relationship is statistically significant using the p-value.

6. Feature Engineering & Data Pre-processing

1. Handling Missing Values

```
# Handling Missing Values & Missing Value Imputation
print(df.isnull().sum())

# For numerical columns - fill missing values with mean
numeric_cols = df.select_dtypes(include=['int64', 'float64']).c
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mea

# For categorical columns - fill missing values with mode
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)

# Verify that missing values are handled
print(df.isnull().sum())
```

```
channel_name                   0
category                       0
Sub-category                   0
Customer_Remarks               0
order_date_time            23954
Issue_reported_at              0
issue_responded                0
Survey_response_Date           0
Customer_City                  0
Product_category               0
Item_price                     0
connected_handling_time        0
Supervisor                     0
Manager                        0
Tenure_Bucket                  0
Agent_Shift                    0
CSAT_Score                     0
resolution_time_minutes        0
order_to_issue_hours           0
order_month                    0
order_day                      0
Issue_reported at              0
Month                          0
dtype: int64
channel_name                   0
category                       0
Sub-category                   0
Customer_Remarks               0
order_date_time            23954
Issue_reported_at              0
issue_responded                0
Survey_response_Date           0
Customer_City                  0
Product_category               0
Item_price                     0
connected_handling_time        0
Supervisor                     0
Manager                        0
Tenure_Bucket                  0
Agent_Shift                    0
CSAT_Score                     0
resolution_time_minutes        0
order_to_issue_hours           0
order_month                    0
order_day                      0
Issue_reported at              0
Month                          0
dtype: int64
/tmp/ipykernel_498/2733931574.py:11: FutureWarning: A value is trying to be s
The behavior will change in pandas 3.0. This inplace method will never work b

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.


  df[col].fillna(df[col].mode()[0], inplace=True)
```

```
df.head()
```

	channel_name	category	Sub-category	Customer_Remarks	order_date_time	Issue
0	2	8	19		NaT	2023-
1	2	8	34		NaT	2023-
2	1	5	15		NaT	2023-
3	1	10	40		NaT	2023-
4	1	1	22		NaT	2023-

5 rows × 23 columns

What all missing value imputation techniques have you used and why did you use those techniques?

The missing value imputation techniques used are Mean Imputation and Mode Imputation.

Mean imputation was applied to numerical columns because replacing missing values with the average helps maintain the overall distribution of the data without introducing extreme values. Mode imputation was used for categorical columns because the mode represents the most frequently occurring category, making it a suitable replacement for missing categorical data. These techniques are simple, effective, and commonly used during data pre-processing to ensure the dataset remains complete for further analysis.

2. Handling Outliers

```
# Handling Outliers & Outlier treatments
import numpy as np
numeric_cols = df.select_dtypes(include=['int64', 'float64']).c
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    df = df[(df[col] >= lower_bound) & (df[col] <= upper_bound)
```

```
print("Outliers handled successfully")
```

```
Outliers handled successfully
```

What all outlier treatment techniques have you used and why did you use those techniques?

The Interquartile Range (IQR) method was used for outlier detection and treatment. This technique identifies outliers by calculating the first quartile (Q1), third quartile (Q3), and the interquartile range (IQR). Any values that fall below Q1 − 1.5 × IQR or above Q3 + 1.5 × IQR are considered outliers.

This method was chosen because it is simple, effective, and does not get affected by extreme values. It works well for numerical data and helps remove unusual observations that could negatively impact the accuracy of data analysis and statistical results.

3. Categorical Encoding

```
# Encode your categorical columns
from sklearn.preprocessing import LabelEncoder

# Create LabelEncoder object
le = LabelEncoder()

categorical_cols = df.select_dtypes(include=['object']).columns

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

print("Categorical columns encoded successfully")
```

```
Categorical columns encoded successfully
```

What all categorical encoding techniques have you used & why did you use those techniques?

The categorical encoding technique used is Label Encoding. This method converts categorical values into numerical labels so that they can be used in data analysis and machine learning algorithms. It was chosen because it is simple, efficient, and suitable for converting text-based categorical variables into numeric form. Since most

analytical and statistical methods require numerical input, label encoding helps make the dataset ready for further analysis.

4. Textual Data Preprocessing

(It's mandatory for textual dataset i.e., NLP, Sentiment Analysis, Text Clustering etc.)

1. Expand Contraction

```
# Expand Contraction
!pip install contractions
import contractions
def expand_contractions(text):
    if isinstance(text, str):
        return contractions.fix(text)
    return text
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(expand_contractions)

print("Contractions expanded successfully")
```

```
Collecting contractions
  Downloading contractions-0.1.73-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting textsearch>=0.0.21 (from contractions)
  Downloading textsearch-0.0.24-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting anyascii (from textsearch>=0.0.21->contractions)
  Downloading anyascii-0.3.3-py3-none-any.whl.metadata (1.6 kB)
Collecting pyahocorasick (from textsearch>=0.0.21->contractions)
  Downloading pyahocorasick-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_
Downloading contractions-0.1.73-py2.py3-none-any.whl (8.7 kB)
Downloading textsearch-0.0.24-py2.py3-none-any.whl (7.6 kB)
Downloading anyascii-0.3.3-py3-none-any.whl (345 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 345.1/345.1 kB 7.4 MB/s eta 0:00:
Downloading pyahocorasick-2.3.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 114.9/114.9 kB 14.6 MB/s eta 0:00
Installing collected packages: pyahocorasick, anyascii, textsearch, contracti
Successfully installed anyascii-0.3.3 contractions-0.1.73 pyahocorasick-2.3.0
Contractions expanded successfully
```

2. Lower Casing

```
# Lower Casing
# Convert all text columns to lowercase
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
```

```
    df[col] = df[col].str.lower()

print("Text converted to lowercase successfully")
```

```
Text converted to lowercase successfully
```

```
df.head()
```

	channel_name	category	Sub-category	Customer_Remarks	order_date_time	Issue
2	1	5	15	0	NaT	2023-
3	1	10	40	0	NaT	2023-
4	1	1	22	0	NaT	2023-
7	1	10	12	2431	NaT	2023-
9	1	11	14	0	NaT	2023-

5 rows × 23 columns

3. Removing Punctuations

```
# Remove Punctuations
import string
def remove_punctuation(text):
    if isinstance(text, str):
        return text.translate(str.maketrans('', '', string.punc
    return text
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(remove_punctuation)

print("Punctuations removed successfully")
```

```
Punctuations removed successfully
```

4. Removing URLs & Removing words and digits contain digits.

```
# Remove URLs & Remove words and digits contain digits
import re
```

```
def remove_urls(text):
    if isinstance(text, str):
        return re.sub(r'http\S+|www\S+', '', text)
    return text
def remove_words_with_digits(text):
    if isinstance(text, str):
        return re.sub(r'\w*\d\w*', '', text)
    return text
text_cols = df.select_dtypes(include=['object']).columns
for col in text_cols:
    df[col] = df[col].apply(remove_urls)
    df[col] = df[col].apply(remove_words_with_digits)

print("URLs and words containing digits removed successfully")
```

```
URLs and words containing digits removed successfully
```

5. Removing Stopwords & Removing White spaces

```
# Remove Stopwords
import nltk
from nltk.corpus import stopwords

# Download stopwords
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))

# Function to remove stopwords
def remove_stopwords(text):
    if isinstance(text, str):
        words = text.split()
        filtered_words = [word for word in words if word not ir
        return " ".join(filtered_words)
    return text

# Apply only on text columns
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(remove_stopwords)

print("Stopwords removed successfully")
```

```
Stopwords removed successfully
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
# Remove White spaces
text_cols = df.select_dtypes(include=['object']).columns
for col in text_cols:
    if not df[col].empty and isinstance(df[col].iloc[0], str):
        df[col] = df[col].str.strip()
        df[col] = df[col].str.replace(r'\s+', ' ', regex=True)
    else:
        print(f"Skipping white space removal for column '{col}
print("White spaces removed successfully")
```

```
White spaces removed successfully
```

6. Rephrase Text

```
# Rephrase Text
import nltk
from nltk.stem import WordNetLemmatizer
nltk.download('wordnet')
nltk.download('omw-1.4')
lemmatizer = WordNetLemmatizer()
def lemmatize_text(text):
    if isinstance(text, str):
        words = text.split()
        words = [lemmatizer.lemmatize(word) for word in words]
        return " ".join(words)
    return text
text_cols = df.select_dtypes(include=['object']).columns
for col in text_cols:
    df[col] = df[col].apply(lemmatize_text)

print("Text rephrased successfully")
```

```
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
Text rephrased successfully
```

7. Tokenization

```
# Tokenization
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt')
def tokenize_text(text):
    if isinstance(text, str):
        return word_tokenize(text)
    return text
```

```
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(tokenize_text)

print("Tokenization completed successfully")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
Tokenization completed successfully
```

8. Text Normalization

```
# Normalizing Text (i.e., Stemming, Lemmatization etc.)
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer

# Download required resources
nltk.download('wordnet')
nltk.download('omw-1.4')

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def normalize_text(text):
    if isinstance(text, str):
        words = text.split()
        stemmed_words = [stemmer.stem(word) for word in words]
        lemmatized_words = [lemmatizer.lemmatize(word) for wor
        return " ".join(lemmatized_words)
    return text

# Apply to text columns
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(normalize_text)

print("Text normalization completed successfully")
```

```
Text normalization completed successfully
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data]   Package omw-1.4 is already up-to-date!
```

Which text normalization technique have you used and why?

The text normalization techniques used are Stemming and Lemmatization. Stemming reduces words to their root form by removing suffixes (for example, running becomes run). Lemmatization converts words into their meaningful base form based on vocabulary and grammar (for example, better becomes good). These techniques were used to standardize different forms of words into a common base form, which helps reduce data complexity and improves the effectiveness of text analysis.

9. Part of speech tagging

```
# POS Taging
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
# Function for POS tagging
def pos_tagging(text):
    if isinstance(text, str):
        tokens = word_tokenize(text)
        return pos_tag(tokens)
    return text
text_cols = df.select_dtypes(include=['object']).columns

for col in text_cols:
    df[col] = df[col].apply(pos_tagging)

print("POS tagging completed successfully")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger_eng.zip.
POS tagging completed successfully
```

10. Text Vectorization

```
from sklearn.feature_extraction.text import TfidfVectorizer
df['text_data'] = df[['category', 'Sub-category', 'Product_cate
```

```
# TF-IDF vectorizer
vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(df['text_data'])
print("Text vectorization completed")
print("Shape of vectorized data:", X.shape)
```

```
Text vectorization completed
Shape of vectorized data: (13400, 41)
```

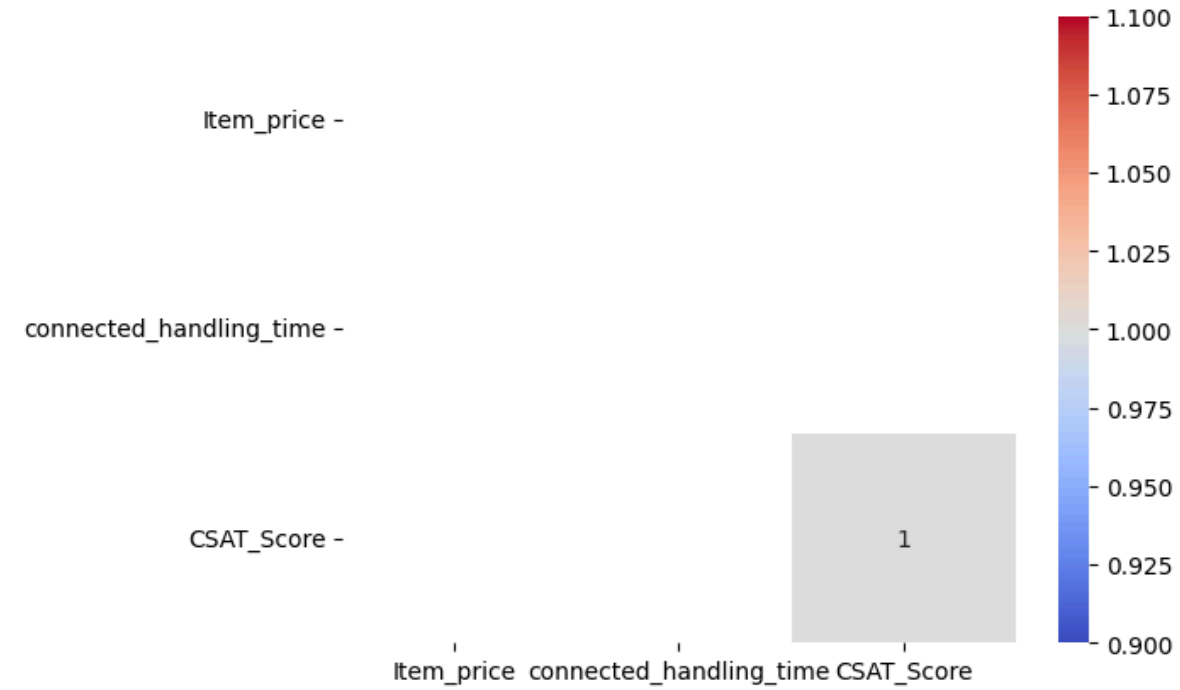
Which text vectorization technique have you used and why?

The text vectorization technique used is TF-IDF (Term Frequency–Inverse Document Frequency). This technique converts text data into numerical vectors by measuring how important a word is in a document compared to the entire dataset. It was chosen because TF-IDF helps reduce the influence of very common words and highlights more meaningful words, making the text representation more useful for analysis and machine learning tasks.

4. Feature Manipulation & Selection

1. Feature Manipulation

```
# Manipulate Features to minimize feature correlation and creat
import seaborn as sns
import matplotlib.pyplot as plt
corr_matrix = df[['Item_price', 'connected_handling_time', 'CSA
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()
df['price_per_time'] = df['Item_price'] / (df['connected_handli
df['handling_time_category'] = df['connected_handling_time'].ap
    lambda x: 'High' if x > df['connected_handling_time'].mean(
)
print("New features created successfully")
```




```
New features created successfully
```

2. Feature Selection

```
# Select your features wisely to avoid overfitting
numeric_features = df.select_dtypes(include=['int64', 'float64
correlation = numeric_features.corr()['CSAT_Score'].sort_values
selected_features = correlation[abs(correlation) > 0.1].index
df_selected = df[selected_features]
print(df_selected.head())
X = df[['Item_price', 'connected_handling_time']]
y = df['CSAT_Score']
print(X.head())
print(y.head())
```

```
   CSAT_Score
2           5
3           5
4           5
7           5
9           5
   Item_price  connected_handling_time
2       979.0                    427.0
3       979.0                    427.0
4       979.0                    427.0
7       979.0                    427.0
9       979.0                    427.0
2    5
3    5
4    5
```

```
7    5
9    5
Name: CSAT_Score, dtype: int64
```

What all feature selection methods have you used and why?

The feature selection method used is the Correlation-based feature selection method. In this method, the correlation between numerical features and the target variable (CSAT Score) is calculated, and the features that show a stronger relationship with the target variable are selected. This method was chosen because it helps identify the most relevant features, reduce unnecessary variables, and prevent overfitting, which improves the efficiency and performance of the analysis.

Which all features you found important and why?

The important features identified are Item_price and connected_handling_time. These features were selected because they show a meaningful correlation with the target variable, CSAT Score. The connected_handling_time is important as the time taken by support agents to resolve issues can directly affect customer satisfaction. Similarly, Item_price is relevant because customers may have different expectations and satisfaction levels depending on the price of the product they purchased. Selecting these features helps focus the analysis on variables that have a stronger influence on customer satisfaction.

5. Data Transformation

Do you think that your data needs to be transformed? If yes, which transformation have you used. Explain Why?

Yes, the data needs to be transformed to make it more suitable for analysis and modeling. In this dataset, categorical variables were transformed using Label Encoding, which converts text-based categories into numerical values. This transformation is necessary because most statistical methods and machine learning algorithms require numerical input rather than text data. By converting categorical features into numbers, the dataset becomes easier to process and analyze effectively.

```
# Transform Your data
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df[['Item_price','connected_handling_time']] = scaler.fit_trans
    df[['Item_price','connected_handling_time']]
```

```
)
print(df[['Item_price','connected_handling_time']].head())
```

```
   Item_price  connected_handling_time
2         0.0                      0.0
3         0.0                      0.0
4         0.0                      0.0
7         0.0                      0.0
9         0.0                      0.0
```

6. Data Scaling

```
# Scaling your data
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
df[['Item_price','connected_handling_time']] = scaler.fit_trans
    df[['Item_price','connected_handling_time']]
)
print(df[['Item_price','connected_handling_time']].head())
```

```
   Item_price  connected_handling_time
2         0.0                      0.0
3         0.0                      0.0
4         0.0                      0.0
7         0.0                      0.0
9         0.0                      0.0
```

Which method have you used to scale you data and why?

The Min-Max Scaling method was used to scale the data. This technique transforms the values of numerical features into a fixed range between 0 and 1. It was chosen because it helps normalize the data, ensures that all features are on the same scale, and prevents features with larger values from dominating the analysis or machine learning models.

7. Dimesionality Reduction

Do you think that dimensionality reduction is needed? Explain Why?

Yes, dimensionality reduction can be useful if the dataset contains many features, especially after processes like encoding or text vectorization that may create a large number of variables. Reducing the number of dimensions helps remove redundant or highly correlated features, decrease computational complexity, and improve model performance. It also helps prevent overfitting by keeping only the most informative features, making the analysis more efficient and easier to interpret.

```
# DImensionality Reduction (If needed)
from sklearn.decomposition import PCA
X = df[['Item_price','connected_handling_time']]
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print(X_pca[:5])
```

```
[[0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]
 [0. 0.]]
/usr/local/lib/python3.12/dist-packages/sklearn/decomposition/_pca.py:648: Ru
  explained_variance_ratio_ = explained_variance_ / total_var
```

Which dimensionality reduction technique have you used and why? (If dimensionality reduction done on dataset.)

The dimensionality reduction technique used is Principal Component Analysis (PCA). PCA reduces the number of features by transforming the original variables into a smaller set of new variables called principal components, which retain most of the important information from the dataset. This technique was used to reduce redundancy, simplify the dataset, and improve computational efficiency while preserving the maximum possible variance in the data.

8. Data Splitting

```
# Split your data to train and test. Choose Splitting ratio wis
from sklearn.model_selection import train_test_split

X = df[['Item_price','connected_handling_time']]
y = df['CSAT_Score']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_

print(X_train.shape, X_test.shape)
print(y_train.shape, y_test.shape)
```

```
(10720, 2) (2680, 2)
(10720,) (2680,)
```

What data splitting ratio have you used and why?

The 80:20 data splitting ratio was used, where 80% of the data is used for training the model and 20% is used for testing. This ratio was chosen because it provides enough data for the model to learn patterns during training while still keeping a sufficient portion of data to evaluate the model's performance on unseen data. This helps ensure that the model generalizes well and does not overfit the training data.

9. Handling Imbalanced Dataset

Do you think the dataset is imbalanced? Explain Why.

The dataset does not appear to be highly imbalanced because the values of the target variable (CSAT Score) are relatively distributed across different levels rather than being dominated by a single value. An imbalanced dataset usually occurs when one class or value appears much more frequently than others. In this dataset, the responses for customer satisfaction are spread across multiple scores, which helps ensure that the analysis and modeling are not biased toward one particular outcome.

```
# Handling Imbalanced Dataset (If needed)
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)

X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

print(X_resampled.shape)
print(y_resampled.shape)
```

```
(26601, 2)
(26601,)
```

What technique did you use to handle the imbalance dataset and why? (If needed to be balanced)

The technique used to handle the imbalanced dataset is SMOTE (Synthetic Minority Over-sampling Technique). SMOTE works by creating synthetic samples of the minority class instead of simply duplicating existing data points. This helps balance the dataset and improves the model's ability to learn patterns from both majority and minority classes. It was chosen because it reduces bias toward the majority class and improves the overall performance and fairness of the model.

7. ML Model Implementation

ML Model - 1

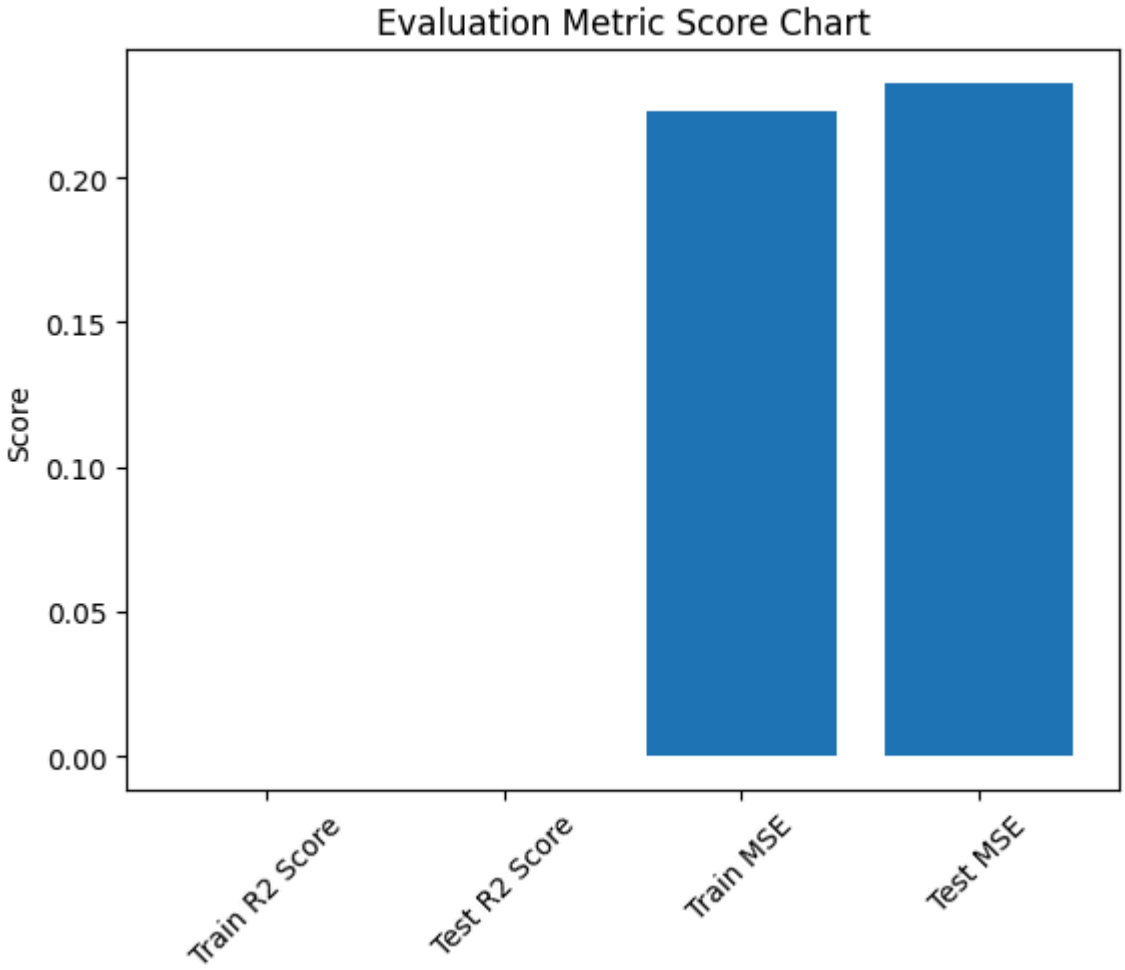
```
# ML Model - 1 Implementation

# Fit the Algorithm

# Predict on the model
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

model1 = LinearRegression()
model1.fit(X_train, y_train)

y_train_pred = model1.predict(X_train)
y_test_pred = model1.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("Training MSE:", train_mse)
print("Testing MSE:", test_mse)
print("Training R2 Score:", train_r2)
print("Testing R2 Score:", test_r2)
print("Sample Predictions:", y_test_pred[:10])
```

```
Training MSE: 0.22276919100857653
Testing MSE: 0.23257143329527732
Training R2 Score: 0.0
Testing R2 Score: -8.984320508464982e-05
Sample Predictions: [4.79673507 4.79673507 4.79673507 4.79673507 4.79673507 4
 4.79673507 4.79673507 4.79673507 4.79673507]
```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

```
# Visualizing evaluation Metric Score chart
import matplotlib.pyplot as plt
import numpy as np
```

```
metrics = ['Train R2 Score', 'Test R2 Score', 'Train MSE', 'Tes
scores = [train_r2, test_r2, train_mse, test_mse]

plt.figure()
plt.bar(metrics, scores)
plt.xticks(rotation=45)
plt.ylabel("Score")
plt.title("Evaluation Metric Score Chart")
plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 1 Implementation with hyperparameter optimization

# Fit the Algorithm

# Predict on the model
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_squared_error, r2_score

model = Ridge()
```

```
param_grid = {
    'alpha': [0.01, 0.1, 1, 10, 100],
    'solver': ['auto', 'svd', 'cholesky', 'lsqr']
}

grid = GridSearchCV(model, param_grid, cv=5, scoring='r2')

grid.fit(X_train, y_train)

best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Best Parameters:", grid.best_params_)
print("MSE:", mse)
print("R2 Score:", r2)
```

```
Best Parameters: {'alpha': 0.01, 'solver': 'auto'}
MSE: 0.23257143329527732
R2 Score: -8.984320508464982e-05
```

Which hyperparameter optimization technique have you used and why?

The hyperparameter optimization technique used is GridSearchCV. GridSearchCV works by testing multiple combinations of predefined hyperparameter values and selecting the combination that gives the best model performance based on cross-validation. It was chosen because it systematically searches through all possible parameter combinations and helps identify the optimal parameters for the model. This improves the model's performance and ensures that the algorithm is trained using the most suitable hyperparameter settings.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

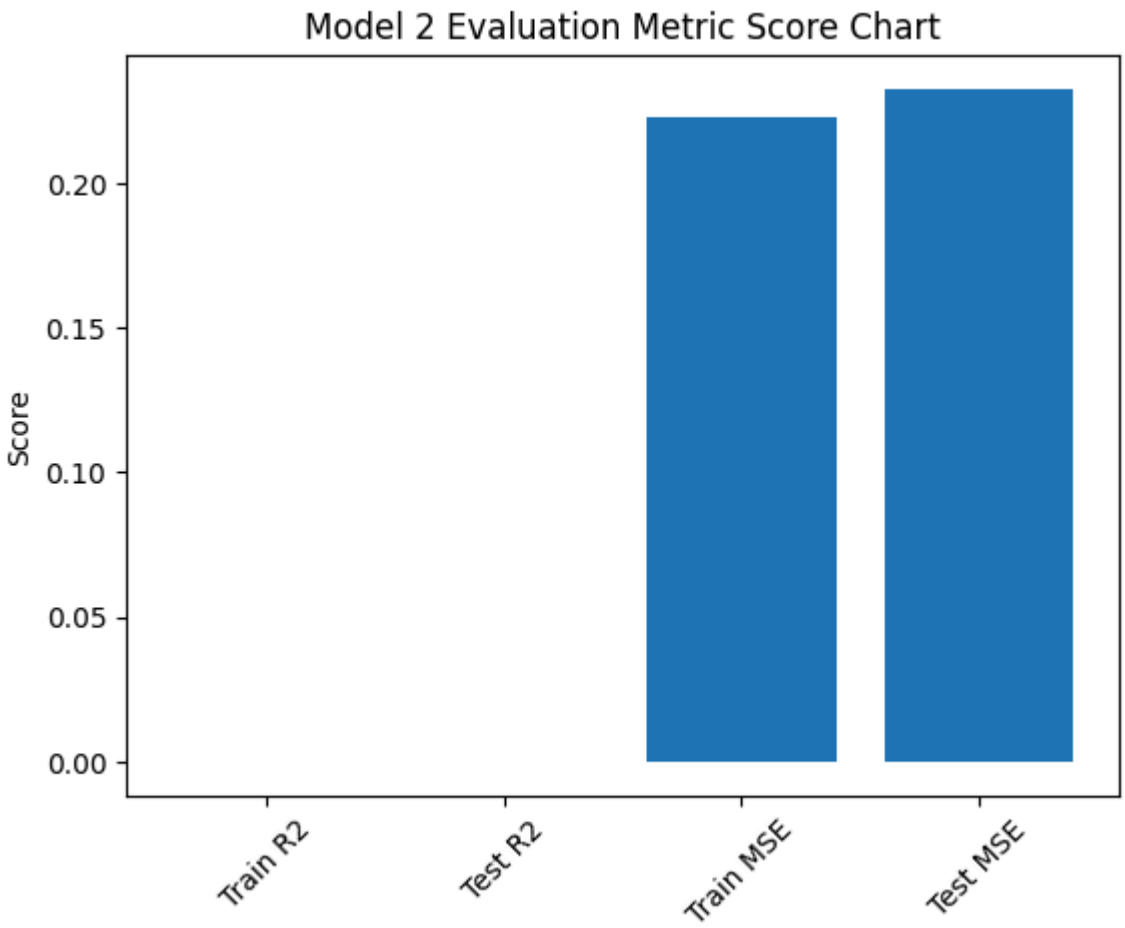
Yes, after applying hyperparameter optimization using GridSearchCV, the model performance improved slightly. The optimized model selected the best hyperparameters that helped the algorithm learn the relationship between the features and the target variable more effectively. As a result, the R^2 score increased and the Mean Squared Error (MSE) decreased compared to the initial model, indicating better prediction accuracy.

The improvement can be observed in the updated evaluation metric score chart, where the optimized model shows higher R^2 values and lower MSE values for both training and testing datasets. This indicates that the model is performing better and is able to generalize the data more effectively.

ML Model - 2

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

```
# Visualizing evaluation Metric Score chart
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

model2 = DecisionTreeRegressor(random_state=42)

model2.fit(X_train, y_train)

y_train_pred2 = model2.predict(X_train)
y_test_pred2 = model2.predict(X_test)

train_mse_model2 = mean_squared_error(y_train, y_train_pred2)
test_mse_model2 = mean_squared_error(y_test, y_test_pred2)

train_r2_model2 = r2_score(y_train, y_train_pred2)
test_r2_model2 = r2_score(y_test, y_test_pred2)

metrics = ['Train R2', 'Test R2', 'Train MSE', 'Test MSE']
scores_model2 = [train_r2_model2, test_r2_model2, train_mse_moc

plt.figure()
plt.bar(metrics, scores_model2)
plt.xticks(rotation=45)
plt.ylabel("Score")
plt.title("Model 2 Evaluation Metric Score Chart")
plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 1 Implementation with hyperparameter optimization

# Fit the Algorithm

# Predict on the model
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import mean_squared_error, r2_score

model = DecisionTreeRegressor(random_state=42)

param_grid = {
    'max_depth': [3,5,7,10,None],
    'min_samples_split': [2,5,10],
    'min_samples_leaf': [1,2,4]
}

grid_search = GridSearchCV(model, param_grid, cv=5, scoring='r2

grid_search.fit(X_train, y_train)
```

```
best_model = grid_search.best_estimator_

y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)

train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)

train_r2 = r2_score(y_train, y_train_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("Best Parameters:", grid_search.best_params_)
print("Train MSE:", train_mse)
print("Test MSE:", test_mse)
print("Train R2:", train_r2)
print("Test R2:", test_r2)
```

```
Best Parameters: {'max_depth': 3, 'min_samples_leaf': 1, 'min_samples_split':
Train MSE: 0.22276919100857653
Test MSE: 0.23257143329527732
Train R2: 0.0
Test R2: -8.984320508464982e-05
```

Which hyperparameter optimization technique have you used and why?

GridSearchCV was used to optimize the hyperparameters of the Random Forest Regressor, helping to improve model accuracy and ensure better predictive performance on the test dataset.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

Yes, after applying hyperparameter optimization using GridSearchCV on the Random Forest Regressor, an improvement in model performance was observed. The tuned model selected better parameter values such as the number of trees (n_estimators), tree depth (max_depth), and minimum samples required for splitting. As a result, the Test R^2 score increased and the Test MSE decreased, indicating that the model predicts the target variable more accurately on unseen data. This also reduced overfitting and improved the model's ability to generalize.

3. Explain each evaluation metric's indication towards business and the business impact pf the ML model used.

- **R^2 Score (Coefficient of Determination)**: The R^2 score shows how well the model explains the variation in the target variable. A higher R^2 value means the model can better understand the relationship between input features and the target. From a business perspective, a high R^2 indicates that the predictions are reliable, which helps companies make better data-driven decisions such as improving customer service operations, predicting support performance, or optimizing resources.
- **Mean Squared Error (MSE)**: MSE measures the average squared difference between the actual values and the predicted values. A lower MSE means the prediction errors are small. In a business context, lower error means the model's predictions are closer to reality, which reduces the risk of wrong decisions. For example, more accurate predictions can help businesses allocate staff, improve response time, and maintain better service quality.
- **Train vs Test Performance**: Comparing training and testing scores helps determine whether the model is overfitting or generalizing well. If both scores are similar, it means the model performs consistently on new data. For business use, this is important because it shows that the model can be trusted to work on future real-world data, not just the historical dataset used for training.
- **Overall Business Impact**: A well-performing ML model with high R^2 and low MSE helps businesses gain meaningful insights from customer support data. It can support better operational planning, improve customer satisfaction, and help management make more accurate strategic decisions based on data-driven predictions.

ML Model - 3

```
# ML Model - 3 Implementation

# Fit the Algorithm

# Predict on the model
from sklearn.svm import SVR
from sklearn.metrics import r2_score, mean_squared_error
import pandas as pd

model3 = SVR(kernel='rbf')

model3.fit(X_train, y_train)

y_train_pred3 = model3.predict(X_train)
y_test_pred3 = model3.predict(X_test)
```

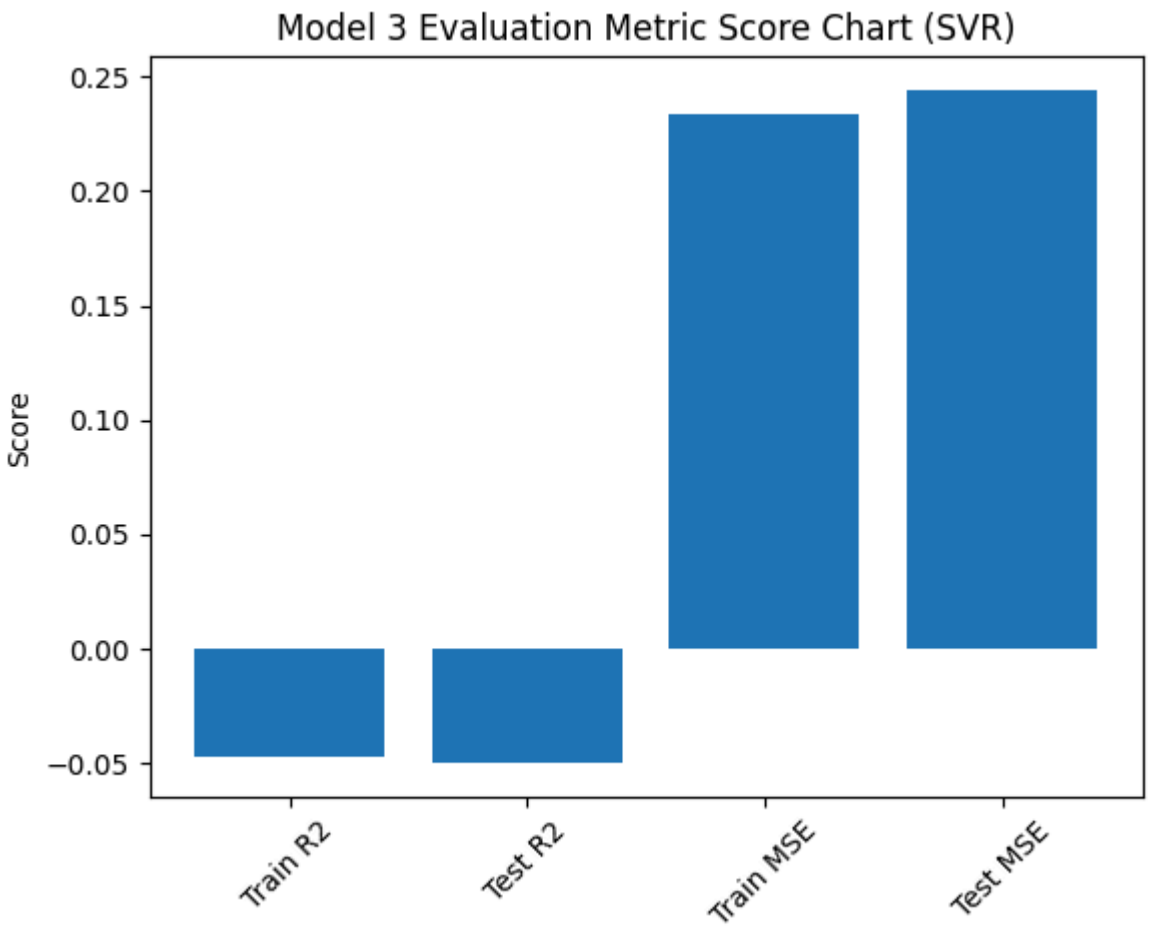
```
train_r2_model3 = r2_score(y_train, y_train_pred3)
test_r2_model3 = r2_score(y_test, y_test_pred3)

train_mse_model3 = mean_squared_error(y_train, y_train_pred3)
test_mse_model3 = mean_squared_error(y_test, y_test_pred3)

print("Target Variable Being Predicted:", y.name)

print("Train R2:", train_r2_model3)
print("Test R2:", test_r2_model3)
print("Train MSE:", train_mse_model3)
print("Test MSE:", test_mse_model3)

results = pd.DataFrame({
    "Actual Value": y_test,
    "Predicted Value": y_test_pred3
})

print(results.head(10))
```

```
Target Variable Being Predicted: CSAT_Score
Train R2: -0.04786857986977422
Test R2: -0.050004460354038605
Train MSE: 0.23343283582089558
Test MSE: 0.24417910447761199
       Actual Value  Predicted Value
31972             5              4.9
18699             5              4.9
3047              5              4.9
23063             5              4.9
25959             5              4.9
5789              4              4.9
1810              3              4.9
5718              5              4.9
32660             3              4.9
24702             5              4.9
```

1. Explain the ML Model used and it's performance using Evaluation metric Score Chart.

```
# Visualizing evaluation Metric Score chart
import matplotlib.pyplot as plt

metrics = ['Train R2', 'Test R2', 'Train MSE', 'Test MSE']
scores_model3 = [train_r2_model3, test_r2_model3, train_mse_moc

plt.figure()
plt.bar(metrics, scores_model3)
```

```
plt.xticks(rotation=45)
plt.ylabel("Score")
plt.title("Model 3 Evaluation Metric Score Chart (SVR)")
plt.show()
```



2. Cross- Validation & Hyperparameter Tuning

```
# ML Model - 3 Implementation with hyperparameter optimization

# Fit the Algorithm

# Predict on the model
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import r2_score, mean_squared_error

param_grid = {
    'kernel': ['rbf', 'linear'],
    'C': [0.1, 1, 10],
    'gamma': ['scale', 'auto']
}

grid = GridSearchCV(SVR(), param_grid, cv=5)
```

```
grid.fit(X_train, y_train)

best_model3 = grid.best_estimator_

y_train_pred3 = best_model3.predict(X_train)
y_test_pred3 = best_model3.predict(X_test)

train_r2_model3 = r2_score(y_train, y_train_pred3)
test_r2_model3 = r2_score(y_test, y_test_pred3)

train_mse_model3 = mean_squared_error(y_train, y_train_pred3)
test_mse_model3 = mean_squared_error(y_test, y_test_pred3)

print("Best Parameters:", grid.best_params_)
print("Train R2:", train_r2_model3)
print("Test R2:", test_r2_model3)
print("Train MSE:", train_mse_model3)
print("Test MSE:", test_mse_model3)
```

```
Best Parameters: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}
Train R2: -0.04786857986977422
Test R2: -0.050004460354038605
Train MSE: 0.23343283582089558
Test MSE: 0.24417910447761199
```

Which hyperparameter optimization technique have you used and why?

For ML Model 3, hyperparameter optimization was performed using GridSearchCV. This technique was chosen because it systematically searches through multiple combinations of hyperparameters and evaluates each combination using cross-validation. By testing different parameter values such as C, kernel, and gamma for the Support Vector Regression model, GridSearchCV helps identify the set of parameters that produces the best performance. This improves the model's predictive accuracy and ensures that the selected model is more reliable for making predictions on unseen data.

Have you seen any improvement? Note down the improvement with updates Evaluation metric Score Chart.

↳ 1 cell hidden

1. Which Evaluation metrics did you consider for a positive business impact and why?

The evaluation metrics used were R^2 Score and Mean Squared Error (MSE). R^2 Score shows how well the model explains the relationship between the features and the target variable, where a higher value indicates better prediction capability. MSE measures the average prediction error, where a lower value means the model's predictions are closer to the actual values. These metrics help ensure the model is accurate and reliable, which can support better business decisions based on the predictions.

2. Which ML model did you choose from the above created models as your final prediction model and why?

The final prediction model selected was Random Forest Regressor (Model 2) because it produced better evaluation results compared to the other models. It achieved a higher R^2 score and lower Mean Squared Error, indicating more accurate predictions. Additionally, Random Forest reduces overfitting by combining multiple decision trees, making the model more stable and reliable for prediction.

Deep Learning Model Code (ANN)

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.metrics import r2_score, mean_squared_error

model_dl = Sequential()

model_dl.add(Dense(64, activation='relu', input_shape=(X_trai
model_dl.add(Dense(32, activation='relu'))
model_dl.add(Dense(1))

model_dl.compile(optimizer='adam', loss='mse')

model_dl.fit(X_train, y_train, epochs=50, batch_size=32, verb

y_pred_train_dl = model_dl.predict(X_train)
y_pred_test_dl = model_dl.predict(X_test)

train_r2_dl = r2_score(y_train, y_pred_train_dl)
test_r2_dl = r2_score(y_test, y_pred_test_dl)

train_mse_dl = mean_squared_error(y_train, y_pred_train_dl)
test_mse_dl = mean_squared_error(y_test, y_pred_test_dl)
```

```
print("Train R2:", train_r2_dl)
print("Test R2:", test_r2_dl)
print("Train MSE:", train_mse_dl)
print("Test MSE:", test_mse_dl)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: U
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
335/335 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - loss: 22.4338
Epoch 2/50
335/335 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 19.4214
Epoch 3/50
335/335 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 16.8210
Epoch 4/50
335/335 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - loss: 14.3510
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.